ЛАБОРАТОРНА РОБОТА № 4

Візуальний аналіз даних та часові ряди — аналіз у двох середовищах

Курс: СК26 «Інтелектуальний аналіз та візуалізація даних» **Кафедра систем штучного інтелекту, Національний університет «Львівська політехніка» Оцінювання:** 4 бали

Визначення варіанта

Заповніть три поля в клітинці нижче й виконайте обидві службові клітинки — номер варіанта, ринок і товар визначаться автоматично.

```
# ── Заповніть три поля ──────────────────────────────────────
SURNAME_NAME = 'Бубельник Юрій'      # прізвище та ім'я
GROUP        = 'ШІ-31'      # шифр групи, наприклад СШІ-31
NUMBER       = 2       # ваш номер за списком групи (1–30)

assert SURNAME_NAME.strip(), 'Впишіть прізвище та ім\'я'
assert GROUP.strip(),        'Впишіть групу'
assert isinstance(NUMBER, int) and 1 <= NUMBER <= 30, 'Номер має бути цілим від 1 до 30'
print('Дані прийнято:', SURNAME_NAME, '|', GROUP, '| номер', NUMBER)
```

```
Дані прийнято: Бубельник Юрій | ШІ-31 | номер 2
```

```
# ── Службова клітинка: визначення варіанта. Змінювати не потрібно ──
import json

VARIANTS = json.load(open('lr4_variants_student.json', encoding='utf-8'))
V = VARIANTS[NUMBER - 1]

MARKET, GOOD, GOOD_UA = V['ринок'], V['товар'], V['товар_укр']

print('=' * 72)
print(f'ВАРІАНТ {V["варіант"]}   ·   {SURNAME_NAME}   ·   {GROUP}')
print('=' * 72)
print()
print(f'   Ринок:  {MARKET}')
print(f'   Товар:  {GOOD_UA}   (у наборі даних: {GOOD})')
print()
print('Далі в роботі користуйтеся змінними MARKET і GOOD —')
print('тоді весь код автоматично працюватиме саме з вашим рядом.')
print()
print('Період: 03.2014 – 12.2017 · очікувана довжина ряду: 46 місяців')
print('Фолдів ретроспективної перевірки: 7 · горизонт: 3 місяці · ARIMA(1, 1, 1)')
```

```
========================================================================
ВАРІАНТ 2   ·   Бубельник Юрій   ·   ШІ-31
========================================================================

   Ринок:  Chernihiv
   Товар:  гречка   (у наборі даних: Buckwheat grits - Retail)

Далі в роботі користуйтеся змінними MARKET і GOOD —
тоді весь код автоматично працюватиме саме з вашим рядом.

Період: 03.2014 – 12.2017 · очікувана довжина ряду: 46 місяців
Фолдів ретроспективної перевірки: 7 · горизонт: 3 місяці · ARIMA(1, 1, 1)
```

 Якщо ці дві клітинки не виконано, усі подальші завдання не мають сенсу: перевіряти роботу будуть **за вашим варіантом**, а не за тим рядом, який ви оберете самі.

Дата здачі

Підпис

Мета роботи

Виконати повний цикл аналізу одного часового ряду **у двох середовищах** — у Python та в Orange — і пояснити, чому однаково названі метрики в них дають різні числа.

Вихідні дані

Файл	Зміст
`ціни_україна_2014_2017.csv`	Роздрібні ціни на 25 товарів по 28 ринках України, щомісяця, 03.2014 – 12.2017
`lr4_variants_student.json`	Розподіл варіантів (зчитується автоматично)
`ЛР4_помилковий_аналіз.ipynb`	Звіт колеги, який треба відрецензувати (завдання 8)

Це той самий набір, з яким ми працювали на лекції 4. **Національне середнє в завданнях не використовується** — ваш ряд це конкретний обласний ринок.

Що здавати

1. **`ЛР4_Прізвище_Варіант.ipynb`** — цей зошит із вашим кодом, графіками та письмовими висновками.
2. **`ряд_для_orange.csv`** — експортований ряд (завдання 9).
3. **`потік_orange.ows`** — збережений потік Orange.
4. **Знімки екрана** з Orange для завдань 10–13 — вставте їх у зошит або додайте окремими файлами.

Правила

- **Код пишете самостійно** — у цьому зошиті навмисно немає заготовок.
- **Користуватися штучним інтелектом дозволено**, але ви зобов'язані заповнити декларацію в кінці зошита.
- **Кожне число має супроводжуватися висновком словами.** Робота, у якій є графіки й таблиці, але немає письмових відповідей на поставлені питання, оцінюється як невиконана.
- Усі обчислення мають бути **відтворюваними**: зошит має виконуватися від початку до кінця командою *Restart & Run All* без помилок.

ЧАСТИНА 1. Аналіз засобами Python

Виконується аудиторно.

Завдання 1. Підготовка ряду

Витягніть із набору ряд свого варіанта — скористайтеся змінними `MARKET` і `GOOD`, які визначилися у службовій клітинці, — і приведіть його до **повної місячної сітки** з березня 2014 по грудень 2017.

У звіті наведіть: 1. скільки місяців у сітці та скільки з них мають вимір; 2. у якому саме місяці пропуск; 3. **чому** цей пропуск виник — і чи він у вашому ряді один.

> **Підказка.** Пропуск є в усіх без винятку рядах набору. Якщо ви знайшли більше одного — перевірте, чи правильно побудована сітка.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt

prices = pd.read_csv('ціни_україна_2014_2017.csv')
```

```
prices['date'] = pd.to_datetime(dict(year=prices['рік'], month=prices['місяць'], day=1))

raw = (prices[(prices['ринок'] == MARKET) & (prices['товар'] == GOOD)]
       .sort_values('date').set_index('date')['ціна'])
assert not raw.index.duplicated().any()           # одна ціна на місяць

grid  = pd.date_range('2014-03-01', '2017-12-01', freq='MS')   # MS = перше число місяця
y_raw = raw.reindex(grid)                         # місяці без виміру -> NaN
y_raw.index.name = 'date'

print('Місяців у сітці:', len(grid))
print('З виміром:', y_raw.notna().sum())
print('Пропуски:', [d.strftime('%m.%Y') for d in y_raw[y_raw.isna()].index])

# Чи це локальна проблема Чернігова, чи місяця немає у всьому наборі?
all_months = set(prices['date'].unique())
print('Місяці сітки, відсутні в УСІХ рядах набору:',
      [d.strftime('%m.%Y') for d in grid if d not in all_months])
```

```
Місяців у сітці: 46
З виміром: 45
Пропуски: ['08.2017']
Місяці сітки, відсутні в УСІХ рядах набору: ['08.2017']
```

Висновок до завдання 1:

Сітка 03.2014–12.2017 містить 46 місяців, вимір є для 45. Пропуск один: серпень 2017. Цей місяць відсутній одночасно в усіх 28 ринках набору, тобто це прогалина на рівні джерела даних (за цей місяць значення не надходили взагалі), а не особливість чернігівського ринку. Пропуск у ряді один, що збігається з підказкою до завдання.

Завдання 2. Заповнення пропуску

Заповніть пропуск. Порівняйте **щонайменше три** способи: перенесення останнього значення, лінійну інтерполяцію та ще один на ваш вибір.

У звіті наведіть: 1. значення, яке дає кожен спосіб; 2. який спосіб ви обрали і **чому саме його**; 3. за якої умови заповнювати пропуск було б **неприпустимо** — наведіть приклад із цього ж набору даних.

> **Підказка.** Третій пункт стосується не вашого ряду, а принципу. Згадайте, які області зникають із набору у 2014 році.

```
m = pd.Timestamp('2017-08-01')
fills = {
    'перенесення останнього (ffill)':     y_raw.ffill(),
    'лінійна інтерполяція':               y_raw.interpolate('linear'),
    'кубічний сплайн (інтерполяційний)':  y_raw.interpolate('spline', order=3, s=0),
}
for name, ser in fills.items():
    print(f'{name:36s} {ser[m]:.4f}')
# для порівняння: без s=0 pandas будує ЗГЛАДЖУВАЛЬНИЙ сплайн, який не проходить через відомі точки
print(f'{"(згладжувальний сплайн pandas, s за замовч.)":36s} {y_raw.interpolate("spline", order=3)[m]:.4f}')

print(pd.DataFrame({k: v['2017-05':'2017-11'] for k, v in fills.items()}).round(3))

y = fills['лінійна інтерполяція']          # далі в роботі працюємо з цим рядом
assert len(y) == 46 and y.isna().sum() == 0

# Контрприклад для п.3: скільки місяців має кожен ринок
cov = prices.groupby('ринок')['date'].agg(['min', 'max', 'nunique'])
print(cov.sort_values('nunique').head(4))
```

```
перенесення останнього (ffill)       24.8500
лінійна інтерполяція                 23.9466
кубічний сплайн (інтерполяційний)    24.1886
(згладжувальний сплайн pandas, s за замовч.) 23.5673
            перенесення останнього (ffill)  лінійна інтерполяція  \
date
```

```
2017-05-01                          26.670               26.670
2017-06-01                          25.597               25.597
2017-07-01                          24.850               24.850
2017-08-01                          24.850               23.947
2017-09-01                          23.043               23.043
2017-10-01                          21.240               21.240
2017-11-01                          20.010               20.010

            кубічний сплайн (інтерполяційний)
date
2017-05-01                            26.670
2017-06-01                            25.597
2017-07-01                            24.850
2017-08-01                            24.189
2017-09-01                            23.043
2017-10-01                            21.240
2017-11-01                            20.010
                     min        max  nunique
ринок
Crimea        2014-03-01 2014-03-01        1
m. Sevastopol 2014-03-01 2014-04-01        2
Luhanska      2014-03-01 2017-12-01       40
Donetska      2014-03-01 2017-12-01       42
```

Висновок до завдання 2:

Три способи дають для серпня 2017 року такі значення: перенесення останнього значення — 24.85 грн, лінійна інтерполяція — 23.95 грн, кубічний інтерполяційний сплайн — 24.19 грн.

Обрано лінійну інтерполяцію. У 2017 році ціна стабільно знижувалася приблизно на 1 грн/міс (травень 26.67 → липень 24.85 → вересень 23.04), і лінійне значення продовжує цю тенденцію, тоді як перенесення останнього значення створює штучне «плато». Сплайн дає 24.19, бо враховує прискорення падіння (угнутість кривої), але це значення залежить від припущення про гладкість ряду, а ряд має різкі злами (12.2014, 07.2016). Крім того, результат залежить від реалізації: стандартний виклик `interpolate('spline')` у pandas будує згладжувальний сплайн і дає 23.57. Лінійна інтерполяція простіша, не має прихованих параметрів і однаково реалізована в Orange. Слід врахувати, що 08.2017 потрапляє в перевірну частину фолду 2 ретроспективної перевірки, тож оцінка цього фолду частково спирається на заповнене значення.

Заповнювати пропуск неприпустимо, коли відсутність даних має змістовну причину, а не випадкову. Приклад із цього набору: ряди Криму (лише 03.2014) і Севастополя (03–04.2014) обриваються після анексії, а Луганська й Донецька області мають 40 і 42 місяці замість 45. «Дорахувати» ці місяці означало б вигадати ціни на ринках, які вийшли з-під спостереження. Інтерполювати можна лише короткий внутрішній пропуск із відомими сусідами з обох боків, причина якого не пов’язана з самим процесом.

Завдання 3. Візуальна розвідка

Побудуйте **не менше трьох** графіків свого ряду, які відповідають на різні питання.

У звіті наведіть: 1. для кожного графіка — яке питання він ставить і **чому обрано саме таку геометрію та шкалу**; 2. одну знахідку, якої ви не очікували до побудови графіків.

> **Підказка.** Графік без пояснення вибору форми не зараховується. Нагадування з лекції: геометрія — це твердження про дані.

```
plt.rcParams.update({'axes.spines.top': False, 'axes.spines.right': False,
                     'axes.grid': True, 'grid.color': '#e1e0d9', 'grid.linewidth': 0.6})
BLUE, RED = '#2a78d6', '#e34948'
GAP = y_raw[y_raw.isna()].index

# — Графік 1. Як змінювалася ціна за весь період? —
fig, ax = plt.subplots(figsize=(11, 3.8))
ax.plot(y.index, y.values, color=BLUE, lw=2)
ax.plot(GAP, y[GAP], 'o', mfc='white', mec=BLUE, ms=8, mew=2, label='заповнено (08.2017)')
for d, txt, off in [('2014-12-01', 'стрибок ×1.86', (-100, 6)), ('2016-07-01', 'пік', (10, -18))]:
```

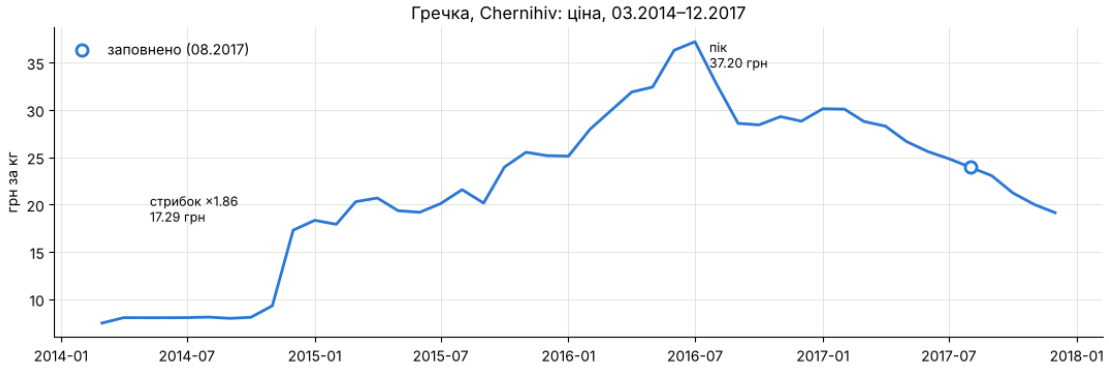
```
    d = pd.Timestamp(d)
    ax.annotate(f'{txt}\n{y[d]:.2f} грн', (d, y[d]), xytext=off, textcoords='offset points', fontsize=9)
ax.set_ylabel('грн за кг'); ax.set_title(f'Гречка, {MARKET}: ціна, 03.2014–12.2017')
ax.legend(frameon=False, loc='upper left'); plt.tight_layout(); plt.show()

# ── Графік 2. Наскільки різкими були щомісячні зміни і коли стались шоки? ──
chg = y.pct_change().dropna() * 100
fig, ax = plt.subplots(figsize=(11, 3.6))
ax.bar(chg.index, chg.values, width=25, color=np.where(chg > 0, RED, BLUE))
ax.axhline(0, color='#898781', lw=0.8)
ax.set_ylabel('зміна до попер. місяця, %')
ax.set_title('Щомісячна зміна ціни (червоне — подорожчання, синє — здешевлення)')
plt.tight_layout(); plt.show()

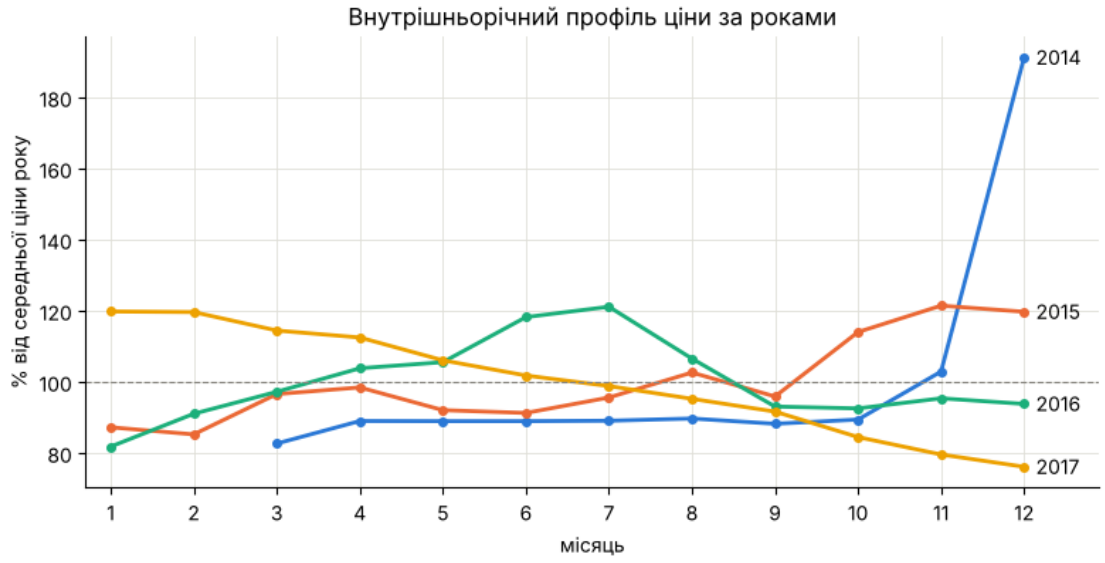
# ── Графік 3. Чи є повторюваний внутрішньорічний патерн? ──
idx = y / y.groupby(y.index.year).transform('mean') * 100   # % від середньої ціни свого року
COLORS = ['#2a78d6', '#eb6834', '#1baf7a', '#eda100']
fig, ax = plt.subplots(figsize=(8, 4.2))
for c, (yr, part) in zip(COLORS, idx.groupby(idx.index.year)):
    ax.plot(part.index.month, part.values, color=c, lw=2, marker='o', ms=4)
    ax.annotate(str(yr), (part.index.month[-1], part.values[-1]), xytext=(6, 0),
                textcoords='offset points', va='center')
ax.axhline(100, color='#898781', lw=0.8, ls='--')
ax.set_xticks(range(1, 13)); ax.set_xlim(0.7, 12.9)
ax.set_xlabel('місяць'); ax.set_ylabel('% від середньої ціни року')
ax.set_title('Внутрішньорічний профіль ціни за роками')
plt.tight_layout(); plt.show()

print('Найбільші зміни, %:'); print(chg.sort_values().iloc[[0, 1, -2, -1]].round(1))
print('Частка місяців зростання:', round((chg > 0).mean(), 2))

# ── Чи стрибок 12.2014 специфічний для гречки? Медіанна зміна 11→12.2014 по обласних ринках для кожного товару ──
regional = prices[prices['ринок'] != 'National Average']
wide = regional.pivot_table(index=['товар', 'ринок'], columns='date', values='ціна')
jump = ((wide[pd.Timestamp('2014-12-01')] / wide[pd.Timestamp('2014-11-01')] - 1) * 100).dropna()
print('\nМедіанна зміна 11→12.2014 по ринках, %:')
print(jump.groupby('товар').median().sort_values(ascending=False).round(1).to_string())
bw = jump.xs(GOOD, level='товар')
print(f'Гречка 11→12.2014 по {len(bw)} ринках: від {bw.min():.0f} % до {bw.max():.0f} %')
```








```
Найбільші зміни, %:
date
2016-09-01   -12.7
2016-08-01   -12.0
2015-10-01    18.9
2014-12-01    85.8
Name: ціна, dtype: float64
Частка місяців зростання: 0.47

Медіанна зміна 11→12.2014 по ринках, %:
товар
Buckwheat grits - Retail              74.5
Cabbage - Retail                      38.8
Beetroots - Retail                    32.0
Carrots - Retail                      17.5
Rice - Retail                         16.0
Onions - Retail                       13.1
Oil (sunflower) - Retail               7.9
Eggs - Retail                          7.8
Potatoes - Retail                      6.7
Pasta - Retail                         3.8
Meat (beef) - Retail                   3.7
Bread (wheat) - Retail                 2.7
Meat (mixed, sausage) - Retail         2.7
Bread (rye) - Retail                   2.6
Wheat flour (first grade) - Retail     2.3
Fuel (petrol-gasoline) - Retail        1.7
Milk - Retail                          1.6
Meat (pork) - Retail                   1.6
Fuel (diesel) - Retail                 1.2
Sour cream - Retail                    1.1
Butter - Retail                        1.1
Fat (salo) - Retail                    1.0
Curd - Retail                          1.0
Sugar - Retail                         0.7
Meat (chicken, whole) - Retail        -0.3
Гречка 11→12.2014 по 23 ринках: від 52 % до 99 %
```

Висновок до завдання 3:

Графік 1 — лінія ціни.

- *Питання:* як ціна змінювалася з 2014 по 2017 рік?
- *Чому лінія:* час неперервний і впорядкований, а лінія між сусідніми місяцями показує траєкторію. Стовпці тут гірші: 46 стовпців дають візуальний шум, і їхня довжина «від нуля» не несе сенсу для цього питання.

- *Чому вісь Y не з нуля:* у лінійному графіку значення кодується положенням точки, а не довжиною, тож нуль не обов'язковий, а важливо бачити форму змін. Для стовпців нуль був би обов'язковим.
- *Чому лінійна шкала:* питання про ціну в гривнях. Логарифмічна шкала вирівняла б відносні зміни, але для цього є графік 2.
- Порожній маркер на 08.2017 показує, що це заповнене, а не виміряне значення.

Графік 2 — стовпці щомісячних змін у %.

- *Питання:* наскільки різкими були зміни і коли були шоки?
- *Чому стовпці:* кожна зміна — окрема подія конкретного місяця. Нуль тут змістовний («ціна не змінилась»), тому довжина від нуля коректна.
- *Чому відсотки, а не гривні:* +1 грн при ціні 8 грн і при ціні 35 грн — різні за силою зміни; відсотки роблять 2014 і 2017 роки порівнянними.
- *Колір:* дві контрастні барви для двох напрямків (подорожчання / здешевлення) і сіра нульова лінія.

Графік 3 — сезонний графік, нормований на середнє року.

- *Питання:* чи повторюється всередині року один і той самий патерн?
- *Геометрія:* вісь X — місяць року (позиція в циклі), по одній лінії на рік. Якби сезонність була, лінії мали б схожу форму.
- *Чому нормування:* без нього лінія 2014 (~8 грн) лежала б унизу, лінія 2016 (~30 грн) вгорі, і форму не можна було б порівняти.
- *Результат:* максимуми припадають на різні місяці (2014 — грудень, 2015 — листопад, 2016 — липень, 2017 — січень). Але нормування прибирає лише різницю рівнів між роками, а не тренд усередині року: у 2017 році ціна весь рік падала, тож весна автоматично вища за осінь, а 2014 рік спотворює грудневий стрибок. Тому форму ліній визначає передусім тренд, і висновку про сезонність із цього графіка зробити не можна. Це питання розв'язує STL-розклад (завдання 4).
- *Обмеження:* 2014 рік неповний (березень–грудень), тому його середнє зміщене.

Неочікувана знахідка. За один місяць, з листопада до грудня 2014 року, ціна зросла на 85.8 % (9.30 → 17.29 грн). Решта місячних змін не перевищує ±19 %. Ряд не зростав плавно, а «перескочив» на новий рівень — це головний кандидат на точку зміни режиму (завдання 8). Того ж місяця гречка різко подорожчала на всіх ринках країни (від +52 % до +99 %), тоді як хліб, пальне, м'ясо й молочні продукти змінились лише від −0.3 до +3.8 % (медіани по ринках), а помітно подорожчали ще тільки овочі, для яких зимове подорожчання сезонне (+13–39 %). Тож це шок саме на ринку гречки; девальвація гривні 2014 року була лише загальним фоном.

Завдання 4. Розкладання на складові

Розкладіть ряд на тренд, сезонну складову та залишок методом STL із періодом 12.

У звіті наведіть: 1. чотирипанельний графік розкладання; 2. числові значення сили сезонності F_S та сили тренду F_T; 3. **висновок: чи є у вашому ряді сезонність**, із посиланням на порогове значення.

> **Підказка.** STL поверне сезонну складову завжди. Ваше завдання — вирішити, чи вона означає щось, а не чи вона є.

```
from statsmodels.tsa.seasonal import STL

res = STL(y, period=12).fit()                # типові параметри: seasonal=7, robust=False
T, S, R = res.trend, res.seasonal, res.resid

# — чотирипанельний графік —
fig, axes = plt.subplots(4, 1, figsize=(11, 8), sharex=True)
for ax, (part, name) in zip(axes, [(y, 'вихідний ряд'), (T, 'тренд T'), (S, 'сезонна S'), (R, 'залишок R')]):
    ax.plot(part.index, part.values, color='#2a78d6', lw=1.8)
```

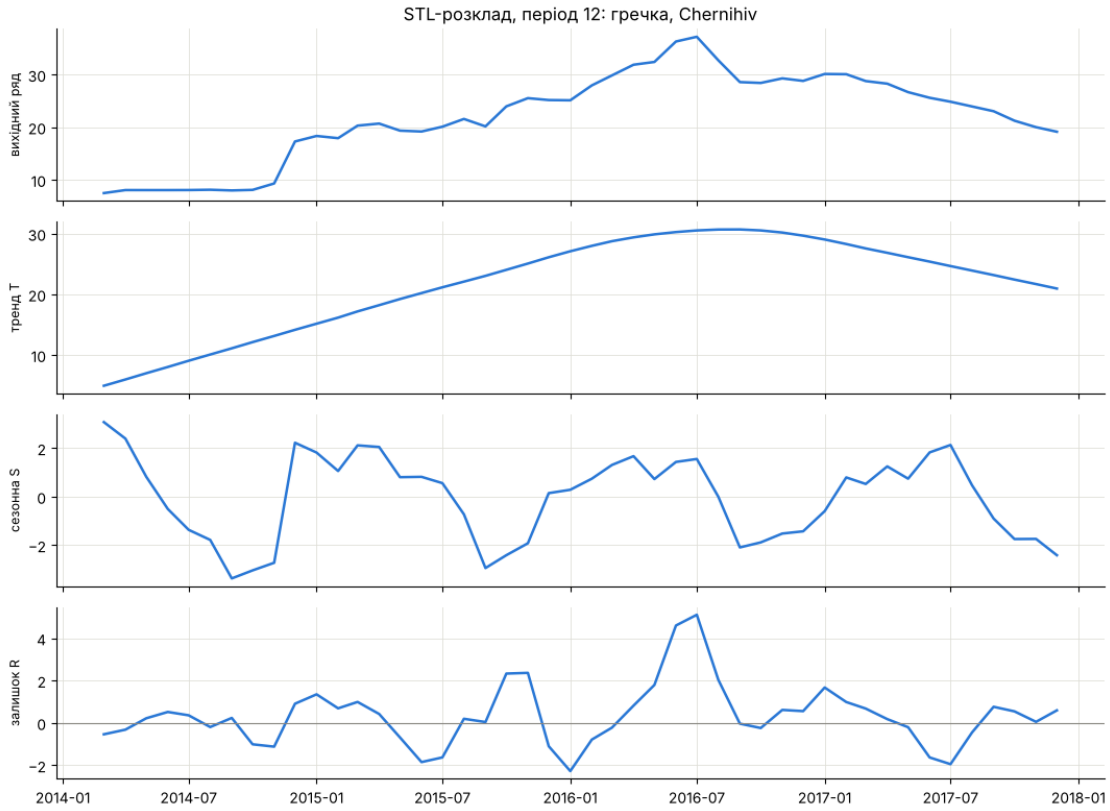
```
    ax.set_ylabel(name, fontsize=9)
axes[3].axhline(0, color='#898781', lw=0.8)
axes[0].set_title(f'STL-розклад, період 12: {GOOD_UA}, {MARKET}')
plt.tight_layout(); plt.show()

# ── сила сезонності й тренду ──
def strength(T, S, R):
    F_S = max(0, 1 - R.var() / (S + R).var())
    F_T = max(0, 1 - R.var() / (T + R).var())
    return F_S, F_T

F_S, F_T = strength(T, S, R)
print(f'Var(R) = {R.var():.2f}   Var(S+R) = {(S+R).var():.2f}   Var(T+R) = {(T+R).var():.2f}')
print(f'F_S = {F_S:.3f}    F_T = {F_T:.3f}')

# ── чи стабільний сезонний профіль з року в рік? (ряд мінус тренд) ──
detr = (y - T).to_frame('v').assign(рік=y.index.year, міс=y.index.month)
print(detr.pivot(index='рік', columns='міс', values='v').round(1))

# ── чутливість F_S до параметрів STL ──
rows = []
for robust in [False, True]:
    for sw in [7, 13]:
        r = STL(y, period=12, seasonal=sw, robust=robust).fit()
        rows.append((robust, sw, *strength(r.trend, r.seasonal, r.resid)))
print(pd.DataFrame(rows, columns=['robust', 'seasonal', 'F_S', 'F_T']).round(3))
```




```
Var(R) = 2.11   Var(S+R) = 5.18   Var(T+R) = 67.76
F_S = 0.592    F_T = 0.969
міс    1    2    3    4    5    6    7    8    9    10   11   12
рік
2014  NaN  NaN  2.5  2.1  1.0  0.0 -1.0 -2.0 -3.1 -4.0 -3.8  3.1
2015  3.2  1.8  3.1  2.5  0.1 -1.0 -1.1 -0.5 -2.9 -0.1  0.4 -1.0
2016 -2.0 -0.0  1.1  2.5  2.5  6.0  6.7  2.0 -2.1 -2.1 -0.9 -0.9
2017  1.1  1.8  1.2  1.4  0.5  0.2  0.2  0.0 -0.1 -1.2 -1.7 -1.8
   robust  seasonal    F_S    F_T
0   False         7  0.592  0.969
1   False        13  0.567  0.968
```

```
2    True         7  0.558  0.957
3    True        13  0.240  0.907
```

Висновок до завдання 4:

STL-розклад (період 12) дає силу тренду $F_T = 0.97$ і силу сезонності $F_S = 0.59$. Тренд домінує в ряді. F_S перевищує поріг 0.3, тож формально сезонність є і вона помірна. Це підтверджується стабільністю патерну: в усіх чотирьох роках ціна в березні–квітні вища за тренд, а у вересні–жовтні нижча, що узгоджується з циклом «збір урожаю наприкінці літа → зниження ціни восени». Водночас оцінка ненадійна: залежно від параметрів STL F_S змінюється від 0.24 до 0.59. Гладкий тренд не відтворює стрибок грудня 2014 року, і частина цього стрибка потрапляє в сезонну складову. Ряд охоплює лише 3.8 сезонного циклу. Висновок: слабка або помірна сезонність, ймовірно, наявна, але для впевненого висновку даних недостатньо.

Завдання 5. Стаціонарність

Побудуйте ACF і PACF вихідного ряду та його першої різниці. Застосуйте тести ADF і KPSS до обох.

У звіті наведіть: 1. графіки ACF і PACF до та після диференціювання; 2. таблицю з p-значеннями обох тестів для обох рядів; 3. висновок про стаціонарність **із зазначенням, узгоджуються тести між собою чи ні**; 4. якщо не узгоджуються — ваше пояснення, чому.

> **Підказка.** Нульові гіпотези ADF і KPSS протилежні. Перш ніж писати висновок, перевірте, що ви не переплутали напрямок.

```
import warnings
from statsmodels.tsa.stattools import adfuller, kpss, acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

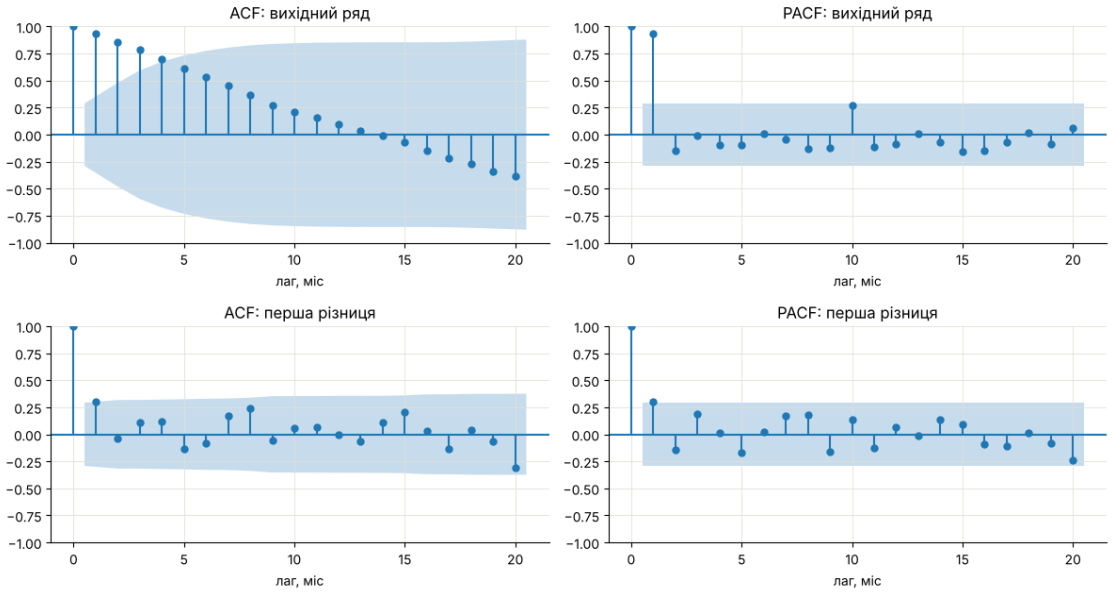
dy = y.diff().dropna()                # перша різниця: Δy_t = y_t − y_{t−1}
LAGS = 20

# ── ACF і PACF до та після диференціювання ──
fig, axes = plt.subplots(2, 2, figsize=(12, 6.5))
plot_acf(y,  lags=LAGS, ax=axes[0, 0], title='ACF: вихідний ряд')
plot_pacf(y, lags=LAGS, ax=axes[0, 1], title='PACF: вихідний ряд', method='ywm')
plot_acf(dy,  lags=LAGS, ax=axes[1, 0], title='ACF: перша різниця')
plot_pacf(dy, lags=LAGS, ax=axes[1, 1], title='PACF: перша різниця', method='ywm')
for ax in axes.flat: ax.set_xlabel('лаг, міс')
plt.tight_layout(); plt.show()

# ── ADF і KPSS для обох рядів ──
def tests(series, reg):
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')          # KPSS обрізає p-value до [0.01, 0.10]; FutureWarning statsmodels
        p_adf = adfuller(series, autolag='AIC', regression=reg)[1]
        p_kpss = kpss(series, regression=reg, nlags='auto')[1]
    return p_adf, p_kpss

rows = []
for name, s in [('вихідний', y), ('перша різниця', dy)]:
    for reg in ['c', 'ct']:
        p_adf, p_kpss = tests(s, reg)
        rows.append({'ряд': name, 'регресія': reg, 'ADF p': round(p_adf, 3), 'KPSS p': round(p_kpss, 3),
                     'ADF каже': 'стаціонарний' if p_adf < 0.05 else 'нестаціонарний',
                     'KPSS каже': 'нестаціонарний' if p_kpss < 0.05 else 'стаціонарний'})
table = pd.DataFrame(rows); print(table.to_string(index=False))

# ── значущі лаги: за тими самими межами, що й смуги на графіках ──
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    for name, s in [('вихідний', y), ('перша різниця', dy)]:
        r, ci   = acf(s, nlags=LAGS, alpha=0.05)                  # ci: межі Бартлетта (розширюються)
        p, ci_p = pacf(s, nlags=LAGS, alpha=0.05, method='ywm')   # ci_p: стала межа ±1.96/√n
        sig_acf  = [k for k in range(1, LAGS + 1) if abs(r[k]) > ci[k, 1] - r[k]]
        sig_pacf = [k for k in range(1, LAGS + 1) if abs(p[k]) > ci_p[k, 1] - p[k]]
        print(f'{name}: ACF значущі лаги {sig_acf}; PACF значущі лаги {sig_pacf}')
```




```
          ряд регресія  ADF p  KPSS p       ADF каже      KPSS каже
     вихідний        c  0.400   0.017 нестаціонарний нестаціонарний
     вихідний       ct  0.993   0.010 нестаціонарний нестаціонарний
перша різниця        c  0.000   0.039   стаціонарний нестаціонарний
перша різниця       ct  0.000   0.100   стаціонарний   стаціонарний
вихідний: ACF значущі лаги [1, 2, 3, 4]; PACF значущі лаги [1]
перша різниця: ACF значущі лаги [1]; PACF значущі лаги [1]
```

Висновок до завдання 5:

Вихідний ряд нестаціонарний: обидва тести це підтверджують (ADF p = 0.40, тобто не відхиляємо H_0 про нестаціонарність; KPSS p = 0.017, тобто відхиляємо H_0 про стаціонарність). ACF спадає повільно (0.94 → 0.70 на 4-му лагу), а PACF має один лаг 0.94. Це типово для ряду з одиничним коренем. Після першого диференціювання ADF вказує на стаціонарність ($p < 0.001$), а KPSS у варіанті зі сталим рівнем ще відхиляє стаціонарність ($p = 0.039$): тести не узгоджуються. Пояснення: середня місячна зміна до піку липня 2016 становить +1.06 грн, а після нього −1.06 грн. Ряд змін не коливається навколо одного рівня, і KPSS реагує саме на це, тоді як ADF перевіряє наявність одиничного кореня. Суперечність посилюють мала довжина ряду (45 різниць) і викид грудня 2014 (+7.99 грн). Якщо дозволити в KPSS лінійний тренд, тести узгоджуються. Отже, одного диференціювання достатньо (d = 1). ACF і PACF першої різниці мають значущий лише перший лаг (0.30), тому двічі диференціювати не потрібно.

Завдання 6. Базисні прогнози та ретроспективна перевірка

Побудуйте **чотири** базисні прогнози: наївний, сезонний наївний, середнє та дрейф. Оцініть їх **ретроспективною перевіркою з рухомим початком**: **7 фолдів по 3 кроки**, навчальна частина розширюється.

У звіті наведіть: 1. таблицю MAE, RMSE і MAPE для всіх чотирьох базисів; 2. який базис виявився найкращим і **чому саме він** — поясніть через властивості вашого ряду; 3. для порівняння: ту саму оцінку, зроблену звичайним випадковим поділом 75/25, і **у скільки разів** вона відрізняється.

> **Підказка.** Кількість фолдів і кроків задана жорстко — у Частині 2 вона має збігтися з налаштуваннями Orange.

```
from sklearn.model_selection import train_test_split

N_FOLDS, H = 7, 3                    # 7 фолдів по 3 кроки — як у Orange
n = len(y)

# ── базисні прогнози ──────────────────────────────────────────
```

```python
# train — відомі значення (Series з датами), dates — місяці, які прогнозуємо.
def forecast(train, dates, method):
    out = []
    for d in dates:
        last_date = train.index[train.index <= d].max()      # останній відомий місяць до d
        if pd.isna(last_date):
            out.append(np.nan); continue
        y_last = train[last_date]
        h = (d.year - last_date.year) * 12 + (d.month - last_date.month)   # скільки кроків уперед
        if method == 'наївний':
            out.append(y_last)
        elif method == 'сезонний наївний':
            out.append(train.asof(d - pd.DateOffset(months=12)))         # той самий місяць рік тому
        elif method == 'середнє':
            out.append(train.mean())
        elif method == 'дрейф':
            T = (train.index[-1].year - train.index[0].year) * 12 + (train.index[-1].month - train.index[0].month)
            slope = (train.iloc[-1] - train.iloc[0]) / T             # середня зміна за місяць
            out.append(y_last + h * slope)
    return np.array(out, dtype=float)

METHODS = ['наївний', 'сезонний наївний', 'середнє', 'дрейф']

def metrics(true, pred):
    ok = ~np.isnan(pred)
    e = true[ok] - pred[ok]
    return {'MAE': np.mean(np.abs(e)),
            'RMSE': np.sqrt(np.mean(e ** 2)),
            'MAPE, %': np.mean(np.abs(e / true[ok])) * 100,
            'n': ok.sum()}

# ── ретроспективна перевірка з рухомим початком ──────────────────
folds = []
for f in range(1, N_FOLDS + 1):
    train_end = n - f * H                        # навчання: перші train_end місяців
    folds.append((train_end, list(range(train_end, train_end + H))))

print('Фолди (навчальна частина розширюється):')
for f, (te, idx) in enumerate(folds, 1):
    print(f'  фолд {f}: навчання 03.2014–{y.index[te-1]:%m.%Y} ({te} міс.)  →  '
          f'перевірка {y.index[idx[0]]:%m.%Y}–{y.index[idx[-1]]:%m.%Y}')

cv_true = {m: [] for m in METHODS}
cv_pred = {m: [] for m in METHODS}
for te, idx in folds:
    train = y.iloc[:te]
    for m in METHODS:
        cv_pred[m].extend(forecast(train, y.index[idx], m))
        cv_true[m].extend(y.values[idx])

cv = pd.DataFrame({m: metrics(np.array(cv_true[m]), np.array(cv_pred[m])) for m in METHODS}).T
print('\nРухомий початок, 7 фолдів × 3 кроки (21 прогноз):')
print(cv.round(2))

# ── для порівняння: випадковий поділ 75/25 ──────────────────────
def random_split_scores(seed):
    tr_idx, te_idx = train_test_split(np.arange(n), test_size=0.25, random_state=seed)
    train = y.iloc[np.sort(tr_idx)]              # навчальні місяці розкидані по всьому періоду
    test_dates = y.index[np.sort(te_idx)]
    true = y[test_dates].values
    return {m: metrics(true, forecast(train, test_dates, m)) for m in METHODS}

rs = pd.DataFrame(random_split_scores(42)).T
print('\nВипадковий поділ 75/25 (random_state=42):')
print(rs.round(2))

# один поділ — це лотерея, тому повторимо 200 разів
many = {m: [] for m in METHODS}
for seed in range(200):
    sc = random_split_scores(seed)
    for m in METHODS: many[m].append(sc[m]['MAE'])
rs_mean = pd.Series({m: np.mean(many[m]) for m in METHODS})
```

```python
cmp = pd.DataFrame({'MAE рухомий початок': cv['MAE'],
                    'MAE випадковий 75/25 (seed 42)': rs['MAE'],
                    'MAE випадковий (сер. 200 поділів)': rs_mean})
cmp['у скільки разів'] = cmp['MAE рухомий початок'] / cmp['MAE випадковий (сер. 200 поділів)']
print('\nПорівняння:')
print(cmp.round(2).to_string())
# детальна таблиця: усі 21 прогноз кожного базису поруч із фактом
rows = []
for f, (te, idx) in enumerate(folds, 1):
    for h, i in enumerate(idx, 1):
        rows.append({'фолд': f, 'місяць': y.index[i].strftime('%m.%Y'), 'h': h, 'ФАКТ': y.iloc[i]})
detail = pd.DataFrame(rows)
for m in METHODS:
    detail[m] = cv_pred[m]
print(detail.sort_values(['фолд', 'h'], ascending=[False, True]).round(2).to_string(index=False))
```

```
Фолди (навчальна частина розширюється):
  фолд 1: навчання 03.2014–09.2017 (43 міс.)  →  перевірка 10.2017–12.2017
  фолд 2: навчання 03.2014–06.2017 (40 міс.)  →  перевірка 07.2017–09.2017
  фолд 3: навчання 03.2014–03.2017 (37 міс.)  →  перевірка 04.2017–06.2017
  фолд 4: навчання 03.2014–12.2016 (34 міс.)  →  перевірка 01.2017–03.2017
  фолд 5: навчання 03.2014–09.2016 (31 міс.)  →  перевірка 10.2016–12.2016
  фолд 6: навчання 03.2014–06.2016 (28 міс.)  →  перевірка 07.2016–09.2016
  фолд 7: навчання 03.2014–03.2016 (25 міс.)  →  перевірка 04.2016–06.2016

Рухомий початок, 7 фолдів × 3 кроки (21 прогноз):
                   MAE  RMSE  MAPE, %     n
наївний           2.21  2.94     8.19  21.0
сезонний наївний  8.14  9.28    29.42  21.0
середнє           8.16  9.83    26.89  21.0
дрейф             2.69  3.66    10.31  21.0

Випадковий поділ 75/25 (random_state=42):
                   MAE   RMSE  MAPE, %     n
наївний           1.44   2.03     6.11  12.0
сезонний наївний  9.99  10.52    38.81   8.0
середнє           8.34   9.68    69.58  12.0
дрейф             1.44   1.81     6.78  12.0


Порівняння:
                  MAE рухомий початок  MAE випадковий 75/25 (seed 42)  MAE випадковий (сер. 200 поділів)  у скільки
разів
наївний                          2.21                            1.44                               1.64
1.35
сезонний наївний                 8.14                            9.99                               9.72
0.84
середнє                          8.16                            8.34                               6.59
1.24
дрейф                            2.69                            1.44                               1.68
1.60
 фолд  місяць  h  ФАКТ  наївний  сезонний наївний  середнє  дрейф
    7 04.2016  1 31.89    29.86             20.69    17.02  30.79
    7 05.2016  2 32.41    29.86             19.35    17.02  31.72
    7 06.2016  3 36.30    29.86             19.18    17.02  32.65
    6 07.2016  1 37.20    36.30             20.10    18.79  37.37
    6 08.2016  2 32.72    36.30             21.57    18.79  38.44
    6 09.2016  3 28.58    36.30             20.16    18.79  39.51
    5 10.2016  1 28.42    28.58             23.97    20.15  29.28
    5 11.2016  2 29.29    28.58             25.53    20.15  29.99
    5 12.2016  3 28.81    28.58             25.17    20.15  30.69
    4 01.2017  1 30.12    28.81             25.12    20.92  29.46
    4 02.2017  2 30.08    28.81             27.96    20.92  30.10
    4 03.2017  3 28.78    28.81             29.86    20.92  30.75
    3 04.2017  1 28.28    28.78             31.89    21.63  29.37
    3 05.2017  2 26.67    28.78             32.41    21.63  29.96
    3 06.2017  3 25.60    28.78             36.30    21.63  30.55
    2 07.2017  1 24.85    25.60             37.20    22.02  26.06
    2 08.2017  2 23.95    25.60             32.72    22.02  26.53
    2 09.2017  3 23.04    25.60             28.58    22.02  26.99
    1 10.2017  1 21.24    23.04             28.42    22.15  23.41
    1 11.2017  2 20.01    23.04             29.29    22.15  23.78
    1 12.2017  3 19.14    23.04             28.81    22.15  24.16
```

Висновок до завдання 6:

Чотири базиси оцінено ретроспективною перевіркою з рухомим початком (7 фолдів × 3 кроки, навчальна частина розширюється від 25 до 43 місяців). Найкращий — наївний прогноз: MAE = 2.21 грн, RMSE = 2.94 грн, MAPE = 8.2 %. Дрейф трохи гірший (MAE 2.69): нахил, оцінений за всю історію, лишається додатним (+0.37 грн/міс у останньому фолді), тоді як у 2017 році ціна падала. Сезонний наївний (MAE 8.14) і середнє (8.16) значно гірші, бо в ряді домінує тренд (F_T = 0.97) і рівень змінюється від 8 до 37 грн. Перевага наївного узгоджується з висновком завдання 5: ряд поводиться як випадкове блукання, і остання відома ціна — найкращий простий прогноз.

Випадковий поділ 75/25 дає для наївного прогнозу MAE 1.44 грн (у середньому по 200 поділах 1.64), тобто занижує похибку приблизно в 1.35 раза (для одного поділу з random_state=42 — у 1.5 раза). Причини: навчальна вибірка містить місяці, що йдуть після тестових (витік інформації з майбутнього), а прогноз фактично робиться лише на один крок уперед. Така оцінка не відповідає реальній задачі прогнозування й не придатна для часових рядів.

Завдання 7. Модель ARIMA

Побудуйте модель ARIMA і оцініть її **на тих самих 7 фолдах по 3 кроки**.

Обов'язково використайте порядок **(1, 1, 1)** — саме його ви відтворюватимете в Orange. Додатково визначте, який порядок (p, d, q) ви обрали б **за виглядом ACF і PACF** із завдання 5.

У звіті наведіть: 1. MAE, RMSE і MAPE моделі ARIMA(1,1,1) на рухомому початку; 2. обґрунтування порядку, який ви обрали б самостійно, з посиланням на конкретні лаги; 3. **порівняння з найкращим базисом із завдання 6** — і прямий висновок, чи має ця модель право на існування; 4. графік залишків моделі та висновок, чи лишилася в них структура.

> **Підказка.** Якщо ARIMA програла базису — це законний результат, а не помилка. Напишіть про це прямо.

```
import warnings
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox

def arima_forecast(train, steps, order):
    # Так само, як Orange: statsmodels ARIMA на масиві значень, без константи (для d=1 це типово)
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        res = ARIMA(train.values, order=order).fit()
    return res.get_forecast(steps).predicted_mean

# ── ARIMA на тих самих 7 фолдах × 3 кроки ──
ORDERS = [(1, 1, 1), (1, 1, 0), (0, 1, 1)]
for order in ORDERS:
    name = f'ARIMA{order}'
    cv_true[name], cv_pred[name] = [], []
    for te, idx in folds:
        cv_pred[name].extend(arima_forecast(y.iloc[:te], H, order))
        cv_true[name].extend(y.values[idx])

ALL = ['наївний', 'дрейф'] + [f'ARIMA{o}' for o in ORDERS]
res_tab = pd.DataFrame({m: metrics(np.array(cv_true[m]), np.array(cv_pred[m])) for m in ALL}).T
print('Рухомий початок, 7 фолдів × 3 кроки:')
print(res_tab.round(2))

# детальна таблиця: додаємо стовпчик ARIMA(1,1,1)
detail['ARIMA(1,1,1)'] = cv_pred['ARIMA(1, 1, 1)']
print(detail.sort_values(['фолд', 'h'], ascending=[False, True]).round(2).to_string(index=False))

# MAE по фолдах: наївний проти ARIMA(1,1,1)
e_n = np.array(cv_true['наївний']) - np.array(cv_pred['наївний'])
e_a = np.array(cv_true['ARIMA(1, 1, 1)']) - np.array(cv_pred['ARIMA(1, 1, 1)'])
```

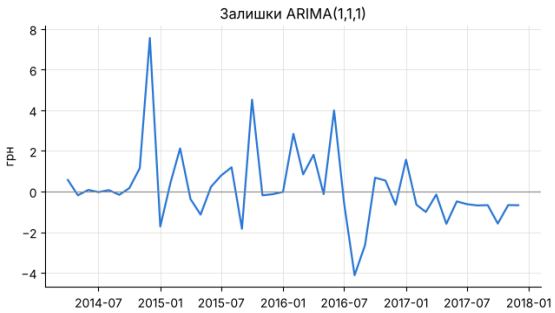
```
print(pd.DataFrame({'фолд': range(1, 8),
                    'наївний': [np.abs(e_n[3*i:3*i+3]).mean() for i in range(7)],
                    'ARIMA(1,1,1)': [np.abs(e_a[3*i:3*i+3]).mean() for i in
range(7)]}).round(2).to_string(index=False))

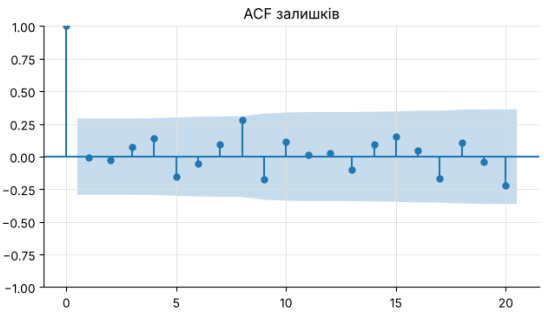
# ── модель на всьому ряді: коефіцієнти та залишки ──
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    full = ARIMA(y.values, order=(1, 1, 1)).fit()
print(full.summary().tables[1])
for o in ORDERS:
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        print(o, 'AIC =', round(ARIMA(y.values, order=o).fit().aic, 1))

resid = pd.Series(full.resid, index=y.index).iloc[1:]   # перший залишок — артефакт старту, відкидаємо
fig, axes = plt.subplots(1, 2, figsize=(13, 3.8))
axes[0].plot(resid.index, resid.values, color='#2a78d6', lw=1.6)
axes[0].axhline(0, color='#898781', lw=0.8)
axes[0].set_title('Залишки ARIMA(1,1,1)'); axes[0].set_ylabel('грн')
plot_acf(resid, lags=20, ax=axes[1], title='ACF залишків')
plt.tight_layout(); plt.show()

print(acorr_ljungbox(resid, lags=[6, 12], return_df=True).round(3))
print('Найбільші залишки:'); print(resid.abs().sort_values(ascending=False).head(3).round(2))
```

```
Рухомий початок, 7 фолдів × 3 кроки:
                MAE  RMSE  MAPE, %     n
наївний        2.21  2.94     8.19  21.0
дрейф          2.69  3.66    10.31  21.0
ARIMA(1, 1, 1)  2.42  3.42     8.87  21.0
ARIMA(1, 1, 0)  2.29  3.08     8.39  21.0
ARIMA(0, 1, 1)  2.26  3.04     8.31  21.0
 фолд  місяць  h  ФАКТ  наївний  сезонний наївний  середнє  дрейф  ARIMA(1,1,1)
    7 04.2016  1 31.89    29.86             20.69    17.02  30.79         29.94
    7 05.2016  2 32.41    29.86             19.35    17.02  31.72         29.92
    7 06.2016  3 36.30    29.86             19.18    17.02  32.65         29.93
    6 07.2016  1 37.20    36.30             20.10    18.79  37.37         37.24
    6 08.2016  2 32.72    36.30             21.57    18.79  38.44         38.18
    6 09.2016  3 28.58    36.30             20.16    18.79  39.51         39.12
    5 10.2016  1 28.42    28.58             23.97    20.15  29.28         27.76
    5 11.2016  2 29.29    28.58             25.53    20.15  29.99         27.83
    5 12.2016  3 28.81    28.58             25.17    20.15  30.69         27.83
    4 01.2017  1 30.12    28.81             25.12    20.92  29.46         28.57
    4 02.2017  2 30.08    28.81             27.96    20.92  30.10         28.60
    4 03.2017  3 28.78    28.81             29.86    20.92  30.75         28.60
    3 04.2017  1 28.28    28.78             31.89    21.63  29.37         28.47
    3 05.2017  2 26.67    28.78             32.41    21.63  29.96         28.52
    3 06.2017  3 25.60    28.78             36.30    21.63  30.55         28.51
    2 07.2017  1 24.85    25.60             37.20    22.02  26.06         25.49
    2 08.2017  2 23.95    25.60             32.72    22.02  26.53         25.50
    2 09.2017  3 23.04    25.60             28.58    22.02  26.99         25.50
    1 10.2017  1 21.24    23.04             28.42    22.15  23.41         22.83
    1 11.2017  2 20.01    23.04             29.29    22.15  23.78         22.85
    1 12.2017  3 19.14    23.04             28.81    22.15  24.16         22.85
 фолд  наївний  ARIMA(1,1,1)
    1     2.91          2.71
    2     1.65          1.55
    3     1.93          1.65
    4     0.87          1.07
    5     0.37          1.03
    6     4.07          5.35
    7     3.68          3.61
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.0912      0.391     -0.233      0.816      -0.858       0.676
ma.L1          0.4621      0.349      1.326      0.185      -0.221       1.145
sigma2         3.5350      0.421      8.396      0.000       2.710       4.360
==============================================================================
(1, 1, 1) AIC = 190.7
(1, 1, 0) AIC = 190.1
(0, 1, 1) AIC = 188.8
```






```
    lb_stat  lb_pvalue
6     2.665      0.850
12   10.226      0.596
Найбільші залишки:
date
2014-12-01    7.57
2015-10-01    4.53
2016-08-01    4.12
dtype: float64
```

Висновок до завдання 7:

ARIMA(1,1,1), оцінена на тих самих 7 фолдах × 3 кроки, дає MAE = 2.42 грн, RMSE = 3.42 грн, MAPE = 8.9 %. Це гірше за наївний прогноз (MAE 2.21, RMSE 2.94, MAPE 8.2 %). ARIMA виграє в чотирьох фолдах із семи, але незначно, і суттєво програє у фолді 6 (липень–вересень 2016): після піку вона продовжила зростання, тоді як ціна різко впала. Коефіцієнти моделі на всьому ряді статистично незначущі (AR: −0.09, p = 0.82; MA: 0.46, p = 0.19). Отже, модель фактично зводиться до випадкового блукання, а наївний прогноз і є ARIMA(0,1,0). Додаткові параметри, оцінені на 25–43 спостереженнях, лише додають похибку. Висновок: ARIMA(1,1,1) для цього ряду не має практичного сенсу, бо простіший наївний прогноз точніший.

Самостійно я обрав би ARIMA(0,1,1): ACF і PACF першої різниці мають єдиний значущий лаг 1 (0.30 при межі 0.29), тож підстав включати одночасно AR- і MA-частину немає. AIC підтверджує вибір (188.8 проти 190.7 для (1,1,1)), але на ретроспективній перевірці і ця модель не перевершує наївного прогнозу (MAE 2.26).

Залишки ARIMA(1,1,1) не мають значущої автокореляції (ACF у межах смуги, тест Льюнга–Бокса p = 0.85 до лагу 6 і 0.60 до лагу 12), тобто лінійної структури не лишилось. Водночас найбільший залишок припадає на стрибок грудня 2014 року (+7.6 грн), мінливість залишків у 2014–2016 роках помітно вища, ніж у 2017-му, а в 2017 році залишки переважно від'ємні. Модель не враховує ні зсуви рівня, ні зміну волатильності.

Завдання 8. Точка зміни режиму

Знайдіть у своєму ряді точку, після якої змінюється **рівень** ряду. Скористайтеся скануванням: для кожної кандидатної дати порівняйте середні до і після та оберіть дату з найбільшою t-статистикою.

У звіті наведіть: 1. графік ряду з позначеною знайденою датою; 2. дату, значення t-статистики, середні рівні до і після; 3. **чи є ця точка значущою** — і якщо ні, так і напишіть; 4. чим знайдена точка відрізняється від звичайного сезонного сплеску у вашому ряді.

> **Підказка.** Не плутайте зсув рівня з сезонним піком. Якщо «аномалії» повторюються щороку в ті самі місяці — це сезон.

```
from scipy import stats

MIN_SEG = 6                                    # мінімум 6 місяців з кожного боку
def scan(series, min_seg=MIN_SEG):
    v = np.asarray(series, dtype=float)
    out = []
    for k in range(min_seg, len(v) - min_seg + 1):  # k — перший місяць «після»
        before, after = v[:k], v[k:]
        t, p = stats.ttest_ind(after, before, equal_var=False)   # t-тест Велча
```

```
        out.append((k, t, p, before.mean(), after.mean()))
    return pd.DataFrame(out, columns=['k', 't', 'p', 'до', 'після'])

sc = scan(y)
sc['дата'] = y.index[sc['k']]
best = sc.loc[sc['t'].abs().idxmax()]
print(f"Точка зміни: {best['дата']:%m.%Y}   t = {best['t']:.2f}   p (наївне) = {best['p']:.1e}")
print(f"Середнє до: {best['до']:.2f} грн ({int(best['k'])} міс.)   після: {best['після']:.2f} грн ({n -
int(best['k'])} міс.)")
print(sc.assign(t=sc['t'].round(2)).nlargest(5, 't')[['дата', 't', 'до', 'після']].to_string(index=False))

# ── графіки: ряд з точкою та t-статистика по всіх кандидатах ──
k0, d0 = int(best['k']), best['дата']
fig, axes = plt.subplots(2, 1, figsize=(11, 6.5), sharex=True, gridspec_kw={'height_ratios': [3, 2]})
ax = axes[0]
ax.plot(y.index, y.values, color='#2a78d6', lw=2, label='ціна')
ax.hlines(best['до'], y.index[0], y.index[k0 - 1], color='#eb6834', lw=2, ls='--', label='середнє до / після')
ax.hlines(best['після'], d0, y.index[-1], color='#eb6834', lw=2, ls='--')
ax.axvline(d0, color='#e34948', lw=1.2)
ax.annotate(f'{d0:%m.%Y}', (d0, y.max()), xytext=(6, -4), textcoords='offset points', color='#e34948')
ax.set_ylabel('грн за кг'); ax.set_title('Точка зміни рівня (сканування t-статистикою)')
ax.legend(frameon=False, loc='upper left')
axes[1].plot(sc['дата'], sc['t'], color='#2a78d6', lw=1.8, marker='o', ms=3)
axes[1].axvline(d0, color='#e34948', lw=1.2)
axes[1].set_ylabel('t-статистика'); axes[1].set_title('t-статистика для кожної кандидатної дати')
plt.tight_layout(); plt.show()

# ── чи значуща? Наївне p не годиться: перевіримо симуляцією на випадкових блуканнях ──
# Ідея: якщо зсуву рівня НЕ було, ряд — просто блукання з такими самими місячними змінами.
# Скануємо тисячі таких блукань і дивимось, як часто max|t| буває не меншим за наш.
rng = np.random.default_rng(0)
steps = y.diff().dropna().values
steps_wo_jump = np.delete(steps, np.argmax(steps))         # без найбільшого стрибка (12.2014)

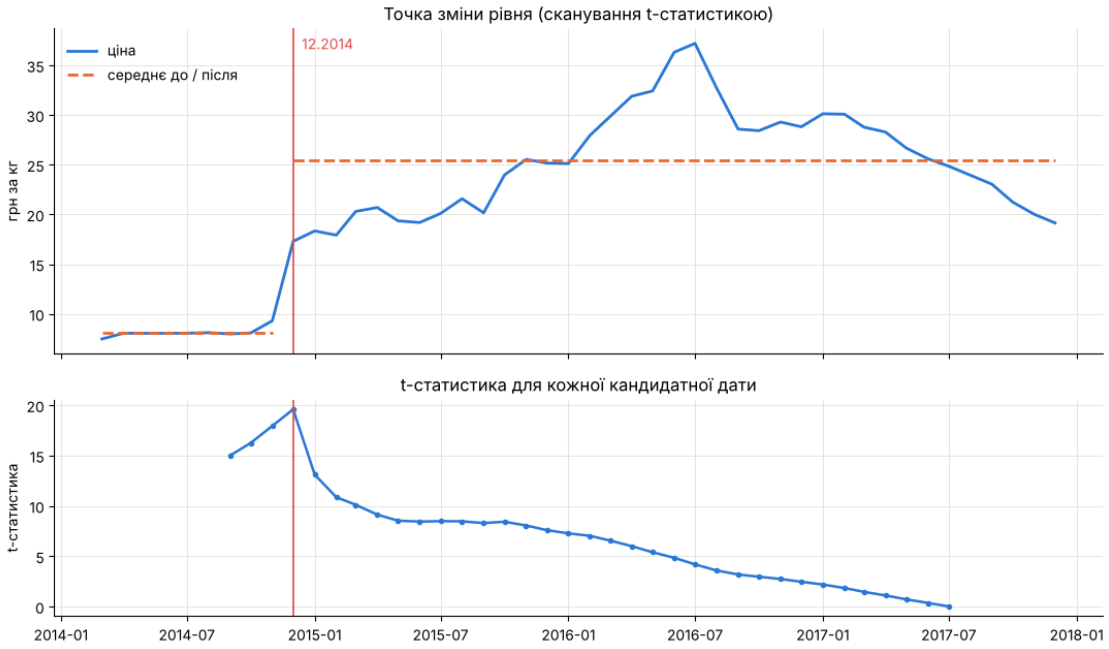
def sim_max_t(step_pool, n_sim=2000):
    res = []
    for _ in range(n_sim):
        walk = np.r_[y.iloc[0], y.iloc[0] + np.cumsum(rng.choice(step_pool, size=n - 1))]
        res.append(scan(walk)['t'].abs().max())
    return np.array(res)

for label, pool in [('кроки ряду, включно зі стрибком', steps), ('кроки ряду без стрибка', steps_wo_jump)]:
    sm = sim_max_t(pool)
    print(f'{label}: медіана max|t| = {np.median(sm):.1f}, 95-й перцентиль = {np.percentile(sm, 95):.1f}, '
          f'p ≈ {np.mean(sm >= abs(best["t"])):.3f}')

# сам стрибок як окрема зміна: наскільки він вибивається з решти місячних змін
z = (steps.max() - steps_wo_jump.mean()) / steps_wo_jump.std(ddof=1)
print(f'Зміна за 12.2014: +{steps.max():.2f} грн; решта змін: середнє {steps_wo_jump.mean():.2f}, '
      f'ст. відх. {steps_wo_jump.std(ddof=1):.2f}; z = {z:.1f}; друга найбільша зміна: +{np.sort(steps)[-2]:.2f}')

# ── чим це не сезонний сплеск: чи повторюється стрибок у той самий місяць інших років ──
chg = y.pct_change() * 100
print('Зміна за листопад→грудень по роках, %:', {yr: round(chg[f'{yr}-12-01'], 1) for yr in [2014, 2015, 2016,
2017]})
```

```
Точка зміни: 12.2014   t = 19.59   p (наївне) = 1.5e-21
Середнє до: 8.13 грн (9 міс.)   після: 25.40 грн (37 міс.)
      дата     t       до     після
2014-12-01 19.59 8.125911 25.401985
2014-11-01 17.94 7.978737 24.978336
2014-10-01 16.25 7.963800 24.545129
2014-09-01 14.99 7.961650 24.130919
2015-01-01 13.10 9.042320 25.627318
```




```
кроки ряду, включно зі стрибком: медіана max|t| = 12.3, 95-й перцентиль = 20.6, p ≈ 0.065

кроки ряду без стрибка: медіана max|t| = 10.8, 95-й перцентиль = 18.1, p ≈ 0.021
Зміна за 12.2014: +7.99 грн; решта змін: середнє 0.08, ст. відх. 1.66; z = 4.8; друга найбільша зміна: +3.89
Зміна за листопад→грудень по роках, %: {2014: np.float64(85.8), 2015: np.float64(-1.4), 2016: np.float64(-1.6),
2017: np.float64(-4.3)}
```

Висновок до завдання 8:

Сканування t-статистикою Велча (мінімум 6 місяців з кожного боку) знаходить точку зміни рівня в грудні 2014 року: t = 19.6, середня ціна до неї 8.13 грн, після 25.40 грн. Наївне p-value (1.5×10^{-21}) не можна вважати коректним: дату обрано як найкращу з 35 кандидатів, а спостереження сильно автокорельовані. Симуляція випадкових блукань з тими самими місячними змінами показує, що таке сканування дає max|t| ≈ 11–12 навіть за відсутності зсуву, а значення 19.6 або більше трапляється з імовірністю 2–7 %. Тож за самою t-статистикою значущість точки межова. Водночас сама зміна за грудень 2014 року (+7.99 грн) відхиляється від решти місячних змін на 4.8 стандартного відхилення й удвічі перевищує наступну найбільшу зміну. Крім того, стрибок одночасно відбувся на всіх 23 обласних ринках (від +52 до +99 %), а серед інших товарів так сильно подорожчали лише овочі, для яких зимове подорожчання сезонне (завдання 3). Тому вважаю цю точку реальною зміною режиму, хоча довести це лише статистикою одного ряду не можна.

Від сезонного сплеску ця точка відрізняється тим, що не повторюється: у 2015–2017 роках зміна листопад→грудень становила від −1.4 до −4.3 %. Крім того, ціна після стрибка не повернулася до попереднього рівня, тобто це постійний зсув, а не тимчасове коливання. Обмеження методу: він припускає сталий рівень після точки, тоді як ціна далі зростала до 37 грн і знижувалась до 19 грн.

Завдання 9. Рецензія на чужий аналіз

Відкрийте `ЛР4_помилковий_аналіз.ipynb`. Це звіт, зроблений на іншому ряді з того самого набору. Він виконується без помилок і виглядає переконливо, але містить **методологічні помилки**.

У звіті наведіть: 1. перелік знайдених помилок — для кожної вкажіть номер клітинки, суть помилки й **чим вона небезпечна**; 2. для трьох найсерйозніших — як саме їх виправити.

 Підказка. Помилок щонайменше вісім. Шукайте не синтаксис, а порушення принципів роботи з часовими рядами.

```
# Перевіряємо твердження колеги на ТОМУ САМОМУ ряді (Вінниця, гречка)
import warnings
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.tsa.arima.model import ARIMA
warnings.filterwarnings('ignore')

def series(market, good):
    return (prices[(prices['ринок'] == market) & (prices['товар'] == good)]
            .sort_values('date').set_index('date')['ціна'])

s_v  = series('Vinnytsia', 'Buckwheat grits - Retail')
beef = series('Vinnytsia', 'Meat (beef) - Retail')
lviv = series('Lviv', 'Buckwheat grits - Retail')
s_vf = s_v.reindex(grid).interpolate()                    # правильно: повна сітка + заповнення

print('[2] пропущені місяці після reindex:', [d.strftime('%m.%Y') for d in grid if d not in s_v.index])
print('[3] максимум ряду: %.2f грн у %s' % (s_v.max(), s_v.idxmax().strftime('%m.%Y')))

for name, x in [('гречка', s_v), ('яловичина', beef)]:
    e = (x - x.shift(1)).dropna()
    print('[4] %-10s середня ціна %5.1f грн | MAE %.2f грн | MAPE %.1f %%' %
          (name, x.mean(), e.abs().mean(), (e.abs() / x).dropna().mean() * 100))

lv = lviv.reindex(grid).interpolate()
print('[5] кореляція рівнів %.3f | кореляція місячних змін %.3f' % (s_vf.corr(lv), s_vf.diff().corr(lv.diff())))

for robust in [False, True]:
    r = STL(s_vf, period=12, robust=robust).fit()
    print('[6] STL robust=%s: F_S = %.2f' % (robust, max(0, 1 - r.resid.var() / (r.seasonal + r.resid).var())))

print('[7] ADF рівень p = %.3f | KPSS рівень p = %.3f | ADF 1-ша різниця p = %.3f' %
      (adfuller(s_vf)[1], kpss(s_vf, nlags='auto')[1], adfuller(s_vf.diff().dropna())[1]))

z = (s_v - s_v.mean()) / s_v.std()
print('[8] «аномалій» при |z| > 1.3: %d,  при |z| > 2 (як написано в тексті): %d' %
      ((z.abs() > 1.3).sum(), (z.abs() > 2).sum()))

e_naive, e_arima = [], []
for f in range(1, 8):                                     # правильна перевірка: 7 фолдів × 3 кроки
    te = len(s_vf) - 3 * f
    train, true = s_vf.iloc[:te], s_vf.values[te:te + 3]
    e_naive += list(true - train.iloc[-1])
    e_arima += list(true - ARIMA(train.values, order=(1, 1, 1)).fit().forecast(3))
print('[9] рухомий початок: MAE наївного %.2f грн | MAE ARIMA(1,1,1) %.2f грн | середня ціна %.1f грн' %
      (np.mean(np.abs(e_naive)), np.mean(np.abs(e_arima)), s_v.mean()))

fc = ARIMA(s_v.values, order=(2, 2, 1)).fit().forecast(12)
print('[10] прогноз (2,2,1): січень 2018 %.2f → грудень 2018 %.2f; остання ціна %.2f' % (fc[0], fc[-1], s_v.iloc[-
1]))

w = s_v['2016-01':'2016-06']
print('[11] 01→06.2016: %.2f → %.2f грн (+%.0f %%); на графіку з віссю від %.2f перший стовпчик майже нульовий' %
      (w.iloc[0], w.iloc[-1], (w.iloc[-1] / w.iloc[0] - 1) * 100, w.min() * 0.995))
```

```
[2] пропущені місяці після reindex: ['08.2017']
[3] максимум ряду: 36.93 грн у 07.2016
[4] гречка     середня ціна  21.3 грн | MAE 1.38 грн | MAPE 6.2 %
[4] яловичина  середня ціна  78.5 грн | MAE 1.51 грн | MAPE 1.9 %
[5] кореляція рівнів 0.996 | кореляція місячних змін 0.937
[6] STL robust=False: F_S = 0.61
[6] STL robust=True: F_S = 0.26
[7] ADF рівень p = 0.348 | KPSS рівень p = 0.018 | ADF 1-ша різниця p = 0.040
[8] «аномалій» при |z| > 1.3: 12,  при |z| > 2 (як написано в тексті): 0
[9] рухомий початок: MAE наївного 2.55 грн | MAE ARIMA(1,1,1) 2.77 грн | середня ціна 21.3 грн

[10] прогноз (2,2,1): січень 2018 17.24 → грудень 2018 8.95; остання ціна 17.67
[11] 01→06.2016: 24.89 → 34.99 грн (+41 %); на графіку з віссю від 24.76 перший стовпчик майже нульовий
```

Висновок до завдання 9:

Кожне ключове твердження колеги перевірено обчисленням на тому самому ряді (Вінниця, гречка) — див. код вище. Номери клітинок наведено так, як їх показує Jupyter (`In [n]`).

#	Клітинка	Суть помилки	Чим небезпечна	Перевірка
1	`In [2]` §1	Пропуски шукаються `isna()` до побудови місячної сітки, тому «пропусків 0»; насправді 45 спостережень замість 46, немає 08.2017	Прихований пропуск: усі методи вважають вересень 2017 наступним після липня, зсуваються лаги й сезонна фаза	після `reindex` бракує 08.2017
2	`In [3]` §2	Висновок про динаміку з двох крайніх точок	Ігнорує траєкторію: між ними пік 36.93 грн (07.2016)	максимум 36.93 у 07.2016
3	`In [3]` §2	«Зниження купівельної спроможності» з номінальної ціни одного товару	Причинний висновок без даних про доходи та інфляцію	—
4	`In [4]` §3	MAE порівнюється між товарами з різним рівнем цін (~21 і ~79 грн)	MAE залежить від масштабу; у відносних похибках висновок протилежний	MAPE: гречка 6.2 %, яловичина 1.9 %
5	`In [5]` §4	«Ціни у Львові визначають ціни у Вінниці» з r = 0.996	Кореляція ≠ причинність; обидва ряди рухає спільний ринок, кореляцію рівнів завищує спільний тренд	кореляція змін 0.94 — зв’язок є, напрямку впливу немає
6	`In [6]` §5	«STL виділив сезонну складову, отже ряд сезонний»	STL повертає її завжди; потрібні F_S і поріг 0.3	з їхнім `robust=True` F_S = 0.26 < 0.3
7	`In [6]` §5	На підставі хибного висновку «надалі використовуємо сезонні моделі»	Ускладнення моделі без підстав	—
8	`In [7]` §6	ADF p = 0.233 витлумачено як «ряд стаціонарний»	Напрямок переплутано: H_0 ADF — нестаціонарність	KPSS p = 0.018 теж: нестаціонарний
9	`In [7]` §6	Сказавши «диференціювання не потрібне», бере різницю двічі «для надійності»	Надмірне диференціювання додає шум; висновки клітинки суперечать один одному	ADF першої різниці p = 0.04 — досить одного разу
10	`In [8]` §7	У тексті поріг 2σ, у коді 1.3σ	Результат підігнано	при 2σ аномалій 0, а не 12
11	`In [8]` §7	z-оцінки від середнього нестаціонарного ряду	«Аномаліями» оголошено низький 2014 рік і пік 2016, тобто тренд; шукати слід у залишках	—
12	`In [9]` §8	Випадковий поділ 75/25, навчальні точки сортуються й подаються в ARIMA як суцільний ряд	Витік інформації з майбутнього; ряд із «дірками» вважається неперервним	—
13	`In [9]` §8	`np.sort(y_te)` порівнюється з прогнозом у часовому порядку; сезонний наївний рахується інакше; звичайного наївного немає	Пари «факт–прогноз» не відповідають одна одній, MAE = 8.73 безглузде, порівняння з базисом нечесне	7×3: наївний 2.55, ARIMA 2.77 грн

#	Клітинка	Суть помилки	Чим небезпечна	Перевірка
14	`In [9]` §8	«Висока точність, усього 8.73 грн»	При середній ціні 21 грн це ~40 % похибки; «прийнятність» без точки відліку	—
15	`In [10]` §9	Порядок раптом змінено на (2,2,1) без обґрунтування; прогноз на 12 місяців без довірчих інтервалів	Неперевірена модель; d = 2 екстраполює нахил із прискоренням	—
16	`In [10]` §9	Висновок «ціна продовжить зростати» суперечить числу	Текст не відповідає результату	прогноз 17.24 → 8.95 грн, остання ціна 17.67
17	`In [11]` §10	Стовпчики з віссю Y від 0.995·min	Зростання на 41 % виглядає як у десятки разів; для стовпчиків вісь має починатися з нуля	24.89 → 34.99 грн
18	Загальні висновки	Повторюють хибні висновки 2, 5, 6, 8, 14, 16	Помилки потрапляють у підсумок	—

Як виправити три найсерйозніші.

1. **Оцінка моделі (`In [9]`)** — замінити випадковий поділ ретроспективною перевіркою з рухомим початком (7 фолдів × 3 кроки, навчання лише на минулому) і порівнювати з наївним прогнозом на тих самих точках. Результат: наївний 2.55 грн, ARIMA(1,1,1) 2.77 грн — модель не краща за найпростіший прогноз.
2. **Стаціонарність (`In [7]`)** — читати ADF правильно (p ≥ 0.05 означає «нестаціонарний»), перевіряти разом із KPSS, тестувати першу різницю й зупинитися, щойно ряд став стаціонарним: d = 1, а не 2.
3. **Прихований пропуск (`In [2]`)** — спершу накласти ряд на повну місячну сітку (`reindex`), потім шукати пропуски (знайдеться 08.2017) і заповнити їх (`interpolate`).

Висновок. У звіті колеги знайдено 18 методологічних помилок. Найсерйозніша — оцінка моделі: випадковий поділ 75/25 дає витік інформації з майбутнього, а порівняння відсортованих фактичних значень із прогнозами в часовому порядку робить MAE = 8.73 безглуздим, тож висновок про «високу точність» хибний. Також переплутано напрямок тесту ADF, застосовано надмірне подвійне диференціювання, а перевірка `isna()` до побудови сітки не виявила відсутній серпень 2017 року. Хибними є й висновки про сезонність, про причинний вплив Львова на Вінницю та про складність прогнозування яловичини. Поріг аномалій у коді не відповідає заявленому, прогноз на 2018 рік суперечить тексту висновку, а стовпчикова діаграма з обрізаною віссю перебільшує зростання.

Завдання 10. Експорт ряду для Orange

Збережіть підготовлений ряд у файл `ряд_для_orange.csv` **рівно у такому вигляді**:

Стовпець	Тип	Зміст
`date`	текст у форматі `РРРР-ММ-ДД`	перше число місяця
`price`	число	ціна, пропуск уже заповнений

Один рядок на місяць, 46 рядків плюс заголовок, роздільник — кома, кодування UTF-8.

У звіті наведіть: 1. підтвердження, що файл має 46 рядків даних і не містить порожніх значень.

 Підказка. Формат заданий жорстко: Orange має отримати **точно ті самі** дані, що й Python, інакше порівняння в Частині 2 втрачає сенс.

```
out = pd.DataFrame({'date': y.index.strftime('%Y-%m-%d'),   # текст РРРР-ММ-ДД, перше число місяця
                    'price': y.values})                     # ціна, пропуск уже заповнений
out.to_csv('ряд_для_orange.csv', index=False, sep=',', encoding='utf-8')

# — перевірка: читаємо файл назад і звіряємо —
check = pd.read_csv('ряд_для_orange.csv', encoding='utf-8', float_precision='round_trip')
print('Стовпці:', list(check.columns))
print('Рядків даних:', len(check))
print('Порожніх значень:', int(check.isna().sum().sum()))
print('Формат дат коректний:', bool(check['date'].str.fullmatch(r'\d{4}-\d{2}-01').all()))
print('Місяці йдуть підряд без пропусків:',
      bool((pd.to_datetime(check['date']).diff().dropna().dt.days.between(28, 31)).all()))
print('Значення збігаються з рядом у Python:', bool(np.array_equal(check['price'].values, y.values)))
print('Серпень 2017 у файлі:', check.loc[check['date'] == '2017-08-01', 'price'].item())
print(check.head(3).to_string(index=False))
```

```
Стовпці: ['date', 'price']
Рядків даних: 46
Порожніх значень: 0
Формат дат коректний: True
Місяці йдуть підряд без пропусків: True
Значення збігаються з рядом у Python: True
Серпень 2017 у файлі: 23.946649999999998
      date  price
2014-03-01 7.4733
2014-04-01 8.0467
2014-05-01 8.0433
```

Висновок до завдання 10:

Ряд збережено у файл ряд_для_orange.csv: два стовпці (date у форматі РРРР-ММ-ДД, перше число місяця; price), роздільник кома, кодування UTF-8. Перевірка зчитуванням файлу назад підтверджує: 46 рядків даних, жодного порожнього значення, усі дати у правильному форматі й ідуть щомісяця без пропусків, значення за серпень 2017 року заповнене (23.95 грн). Значення в файлі точно збігаються з рядом, на якому виконано аналіз у Python. Ціни не округлювались, щоб Orange отримав ті самі дані й розбіжності в Частині 2 могли пояснюватися лише означеннями метрик.

ЧАСТИНА 2. Той самий аналіз засобами Orange

Виконується самостійно.

Потрібен додаток **Timeseries**: у Orange відкрийте *Options → Add-ons…* і встановіть `Orange3-Timeseries`.

Головна умова цієї частини: Orange має отримати **ті самі дані** й **ті самі налаштування перевірки**, що й Python у Частині 1. Тоді будь-яка розбіжність у числах пояснюється **лише означеннями метрик**, а не різними даними чи різними моделями. Саме це вам і треба довести.

Завдання 11. Потік опрацювання даних

Побудуйте в Orange потік:

`File → As Timeseries → Interpolate → Line Chart`

У віджеті **File** завантажте свій `ряд_для_orange.csv`; переконайтеся, що `date` розпізнано як **time**, а `price` — як **numeric**. У віджеті **As Timeseries** оберіть `date` як часову змінну.

У звіті наведіть: 1. знімок екрана з побудованим потоком; 2. знімок `Line Chart` із вашим рядом; 3. підтвердження, що графік у Orange збігається з графіком із завдання 3.

 Підказка. Якщо `date` розпізнано як текст, поверніться до завдання 10 і перевірте формат дати.

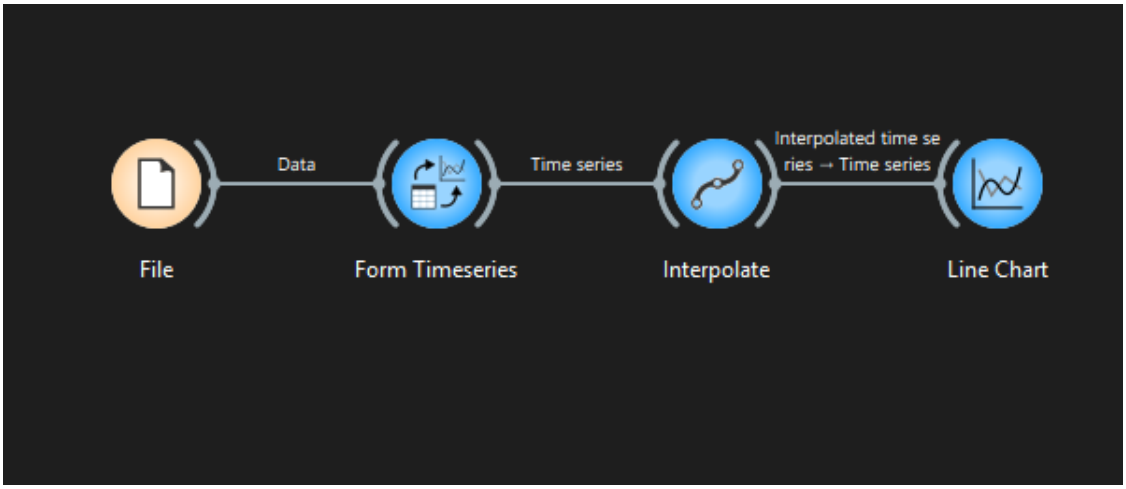
```
# Значення, які показує підказка Line Chart в Orange, звіряємо з рядом у Python
check_points = ['2015-02-01', '2016-07-01', '2017-08-01']
from_file = pd.read_csv('ряд_для_orange.csv', float_precision='round_trip').set_index('date')['price']
for d in check_points:
    print(f'{d}: Python {y[d]:.4f} | файл для Orange {from_file[d]:.4f}')
# Orange (Line Chart, підказка при наведенні): 2015-02-01 → 17.9167
```

```
2015-02-01: Python 17.9167 | файл для Orange 17.9167

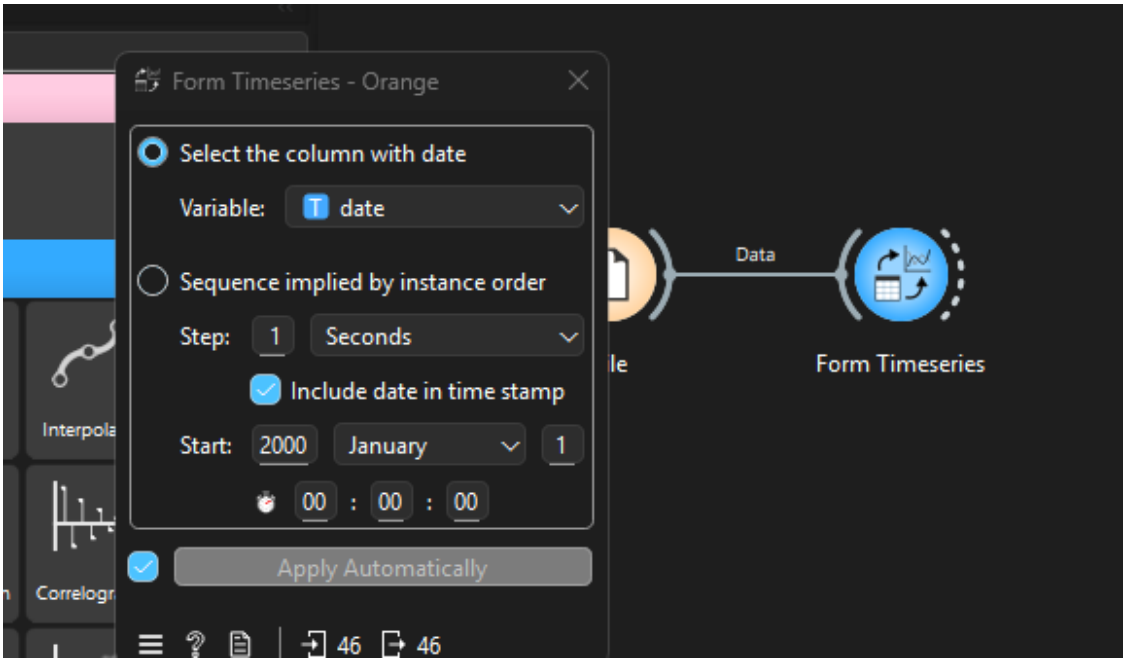

2016-07-01: Python 37.2000 | файл для Orange 37.2000
2017-08-01: Python 23.9466 | файл для Orange 23.9466
```

Знімки екрана Orange (завдання 11)

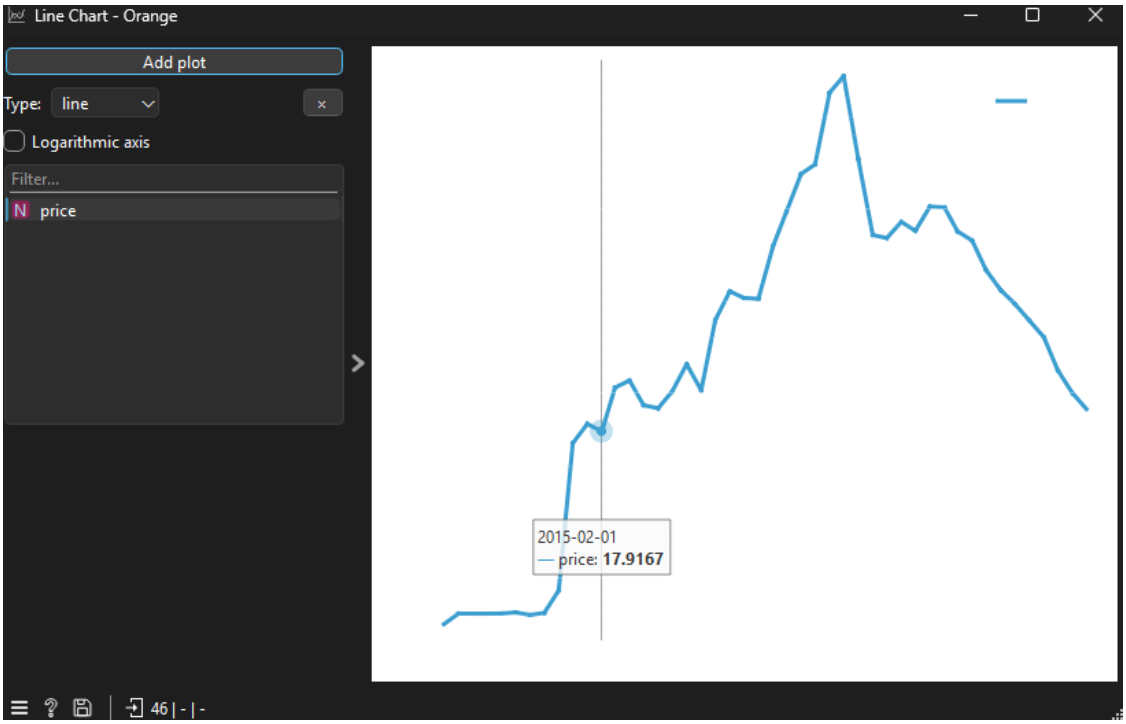
Потік File → Form Timeseries → Interpolate → Line Chart



Form Timeseries: часова змінна — date



Line Chart: ряд у Orange; підказка 2015-02-01 → 17.9167



Висновок до завдання 11:

Потік File → Form Timeseries → Interpolate → Line Chart побудовано. У віджеті File стовпець date розпізнано як datetime, price як numeric (46 рядків, без пропущених значень). У Form Timeseries часовою змінною обрано date. Графік у Line Chart збігається з графіком із завдання 3. Значення в окремих точках ідентичні Python: лютий 2015 року — 17.9167 грн, серпень 2017 року (заповнене значення) — 23.9467 грн. Отже, Orange отримав ті самі дані.

Завдання 12. Автокореляція та диференціювання

Додайте до потоку віджети `Correlogram`, `Difference` та `Seasonal Adjustment`.

У звіті наведіть: 1. знімок корелограми; 2. зіставлення з ACF із завдання 5: **чи збігаються висновки** про значущі лаги; 3. знімок сезонно скоригованого ряду та порівняння з сезонною складовою STL із завдання 4.

> **Підказка.** Orange і statsmodels рахують автокореляцію однаково — якщо картина різна, перевірте, чи однакові дані на вході.

```
# Відтворюємо, що саме малює віджет Correlogram в Orange (Timeseries 0.6.3, functions.py / owcorrelogram.py)
import warnings
from scipy.signal import argrelextrema
from statsmodels.tsa.stattools import acf, pacf

def orange_correlogram(x, use_pacf=False):
    x = np.asarray(x, dtype=float)
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        if use_pacf:
            corr = pacf(x, nlags=min(len(x) // 2 - 1, len(x) - 1), method='ldb')
        else:
            corr = acf(x, nlags=int(.9 * len(x)), fft=True)
    lags = argrelextrema(np.abs(corr), np.greater, order=3)[0]      # лише локальні піки |r|
    vals = corr[lags]
    band = 1.96 * np.sqrt((1 + 2 * (vals ** 2).sum()) / len(x))     # одна стала межа з показаних піків
    return pd.DataFrame({'лаг': lags, 'значення': vals.round(3)}), round(band, 3)
```

```
for title, use_pacf in [('ACF ціни', False), ('PACF ціни', True)]:
    shown, band = orange_correlogram(y, use_pacf)
    print(f'{title}: межа Orange ±{band}')
    print(shown.to_string(index=False))

# для порівняння: межа statsmodels (Бартлетт) на лагу 24 для ACF ціни
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    r, ci = acf(y, nlags=24, alpha=0.05)
print(f'statsmodels: ACF(24) = {r[24]:.3f}, межа Бартлетта на лагу 24 = ±{ci[24, 1] - r[24]:.3f}')
print(f'statsmodels: ACF(1..4) = {np.round(r[1:5], 2)} — у Orange ці лаги не показані')

# перша різниця так, як її бачить Orange: перше значення (NaN) заповнюється найближчим відомим
dy_orange = y.diff().bfill()
for title, use_pacf in [('ACF різниці', False), ('PACF різниці', True)]:
    shown, band = orange_correlogram(dy_orange, use_pacf)
    print(f'{title}: межа Orange ±{band}')
    print(shown.to_string(index=False))
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    r_d, ci_d = acf(y.diff().dropna(), nlags=1, alpha=0.05)
print(f'statsmodels: ACF різниці на лагу 1 = {r_d[1]:.3f}, межа = ±{ci_d[1, 1] - r_d[1]:.3f} — значущий, але в
Orange не показаний')

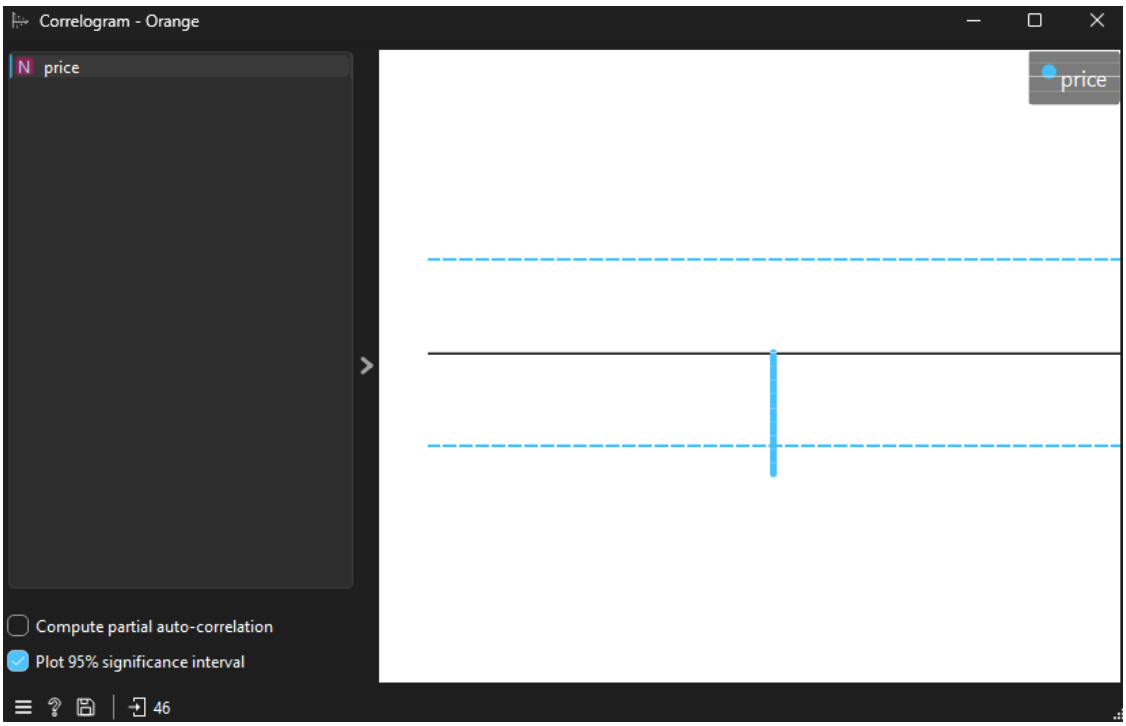
# ── Seasonal Adjustment в Orange = класичний адитивний розклад statsmodels, а не STL ──
from statsmodels.tsa.seasonal import seasonal_decompose, STL
classic = seasonal_decompose(y.values, model='additive', period=12)
s_classic = pd.Series(classic.seasonal, index=y.index)
s_stl = STL(y, period=12).fit().seasonal

prof = pd.DataFrame({'класичний (Orange)': s_classic.groupby(y.index.month).mean(),
                     'STL (сер. по роках)': s_stl.groupby(y.index.month).mean()}).round(2)
prof.index.name = 'місяць'
print(prof.T.to_string())
print('Кореляція профілів:', round(prof.corr().iloc[0, 1], 3))
print('Амплітуда: класичний %.2f грн, STL %.2f грн' % (s_classic.max() - s_classic.min(), s_stl.max() -
s_stl.min()))
print('Класичний: сезонна складова однакова щороку?',
      bool(np.allclose(s_classic['2015'].values, s_classic['2016'].values)))
```

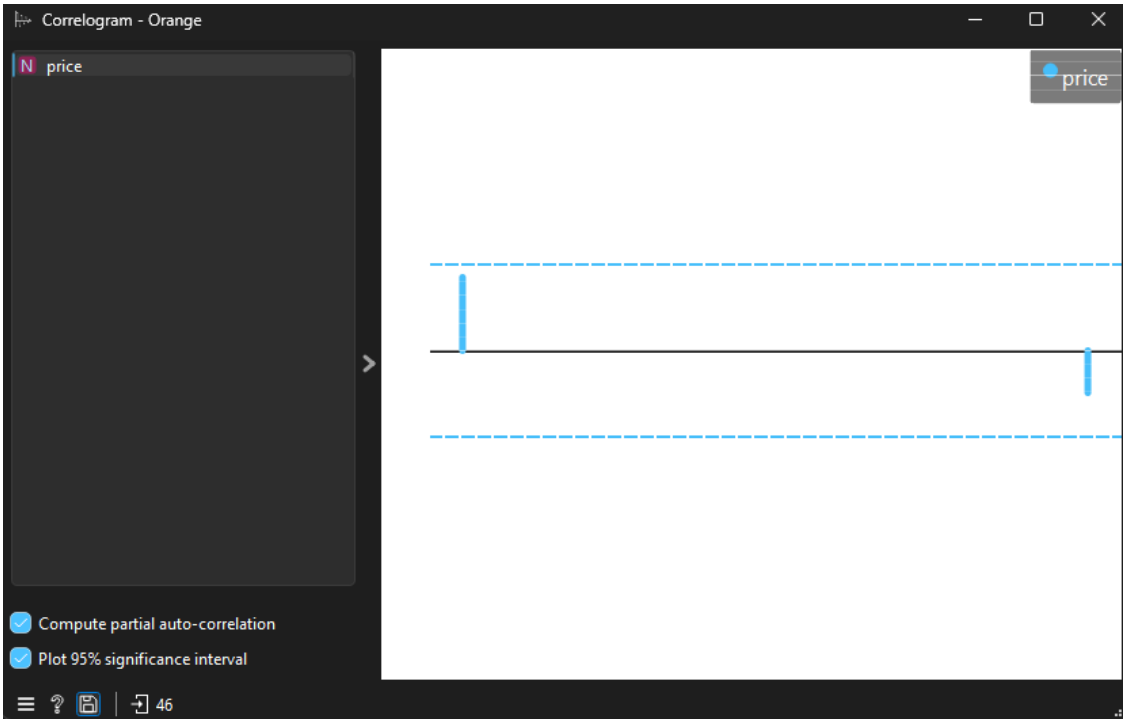
```
ACF ціни: межа Orange ±0.341
 лаг  значення
  24    -0.441
PACF ціни: межа Orange ±0.315
 лаг  значення
  10     0.267
  15    -0.153
statsmodels: ACF(24) = -0.441, межа Бартлетта на лагу 24 = ±0.942
statsmodels: ACF(1..4) = [0.94 0.86 0.78 0.7 ] — у Orange ці лаги не показані
ACF різниці: межа Orange ±0.345
 лаг  значення
   8     0.245
  15     0.204
  20    -0.307
  29    -0.098
  34    -0.085
PACF різниці: межа Orange ±0.319
 лаг  значення
   8     0.180
  14     0.135
  20    -0.241
statsmodels: ACF різниці на лагу 1 = 0.299, межа = ±0.292 — значущий, але в Orange не показаний
місяць                 1     2     3     4     5     6     7     8     9     10    11    12
класичний (Orange)   0.34  0.68  1.24  1.49  0.35  1.06  2.17  0.17 -3.19 -2.51 -1.84  0.04
STL (сер. по роках)  0.50  0.87  1.76  1.84  0.77  0.89  0.72 -0.51 -2.32 -2.27 -1.97 -0.37
Кореляція профілів: 0.928
Амплітуда: класичний 5.36 грн, STL 6.44 грн
Класичний: сезонна складова однакова щороку? True
```

Знімки екрана Orange (завдання 12)

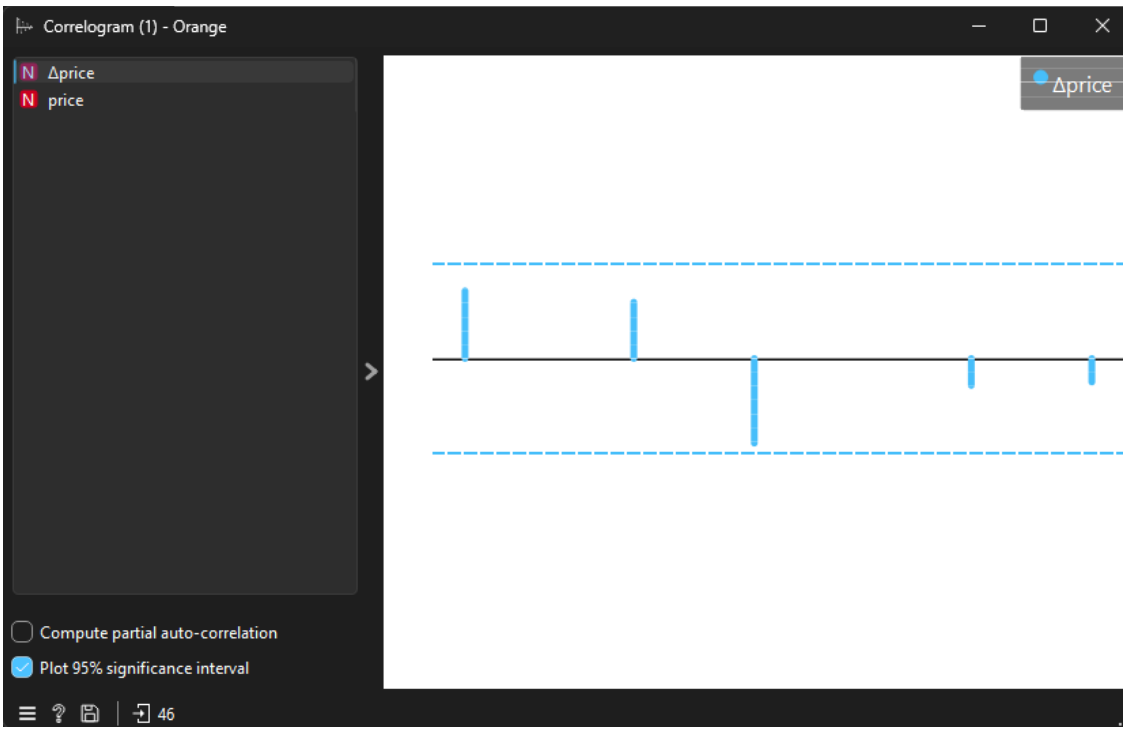
Correlogram: ACF ціни (показано лише лаг 24 = −0.44, межа ±0.34)



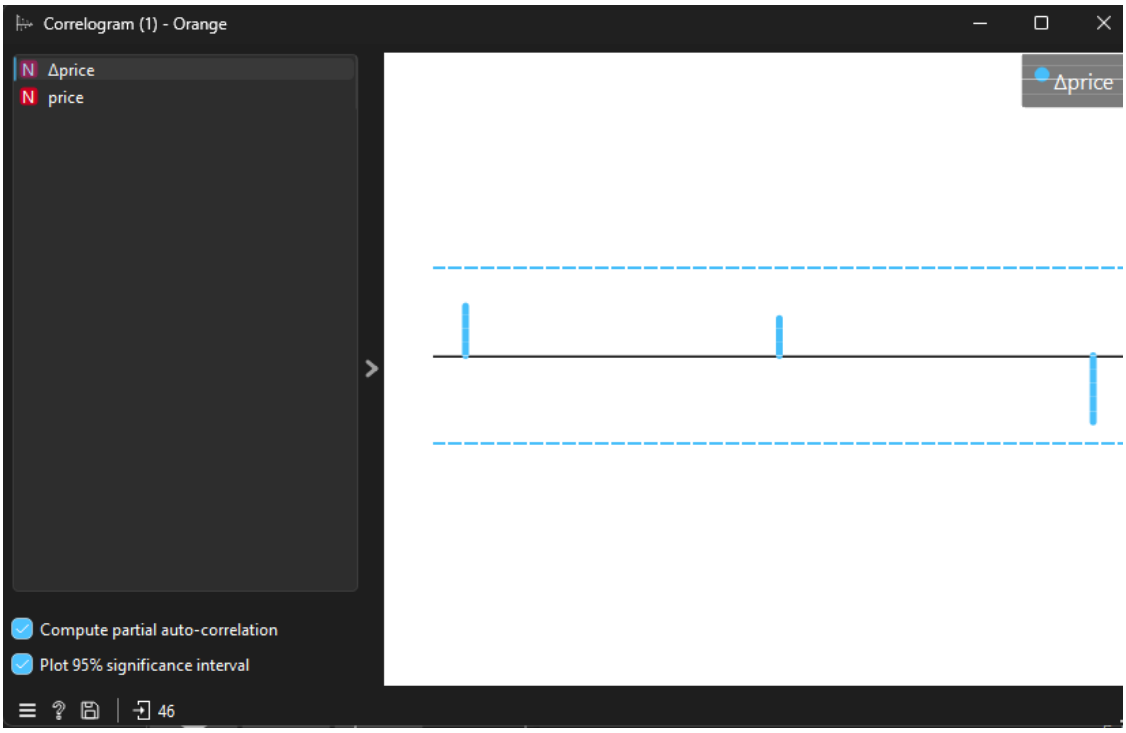
Correlogram: PACF ціни (лаги 10 і 15)



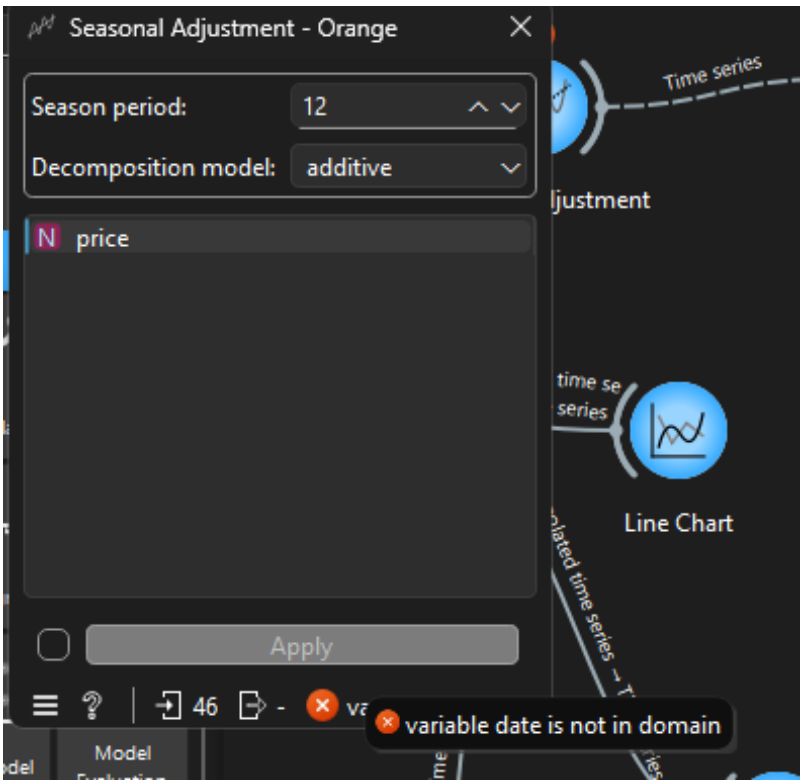
Correlogram: ACF першої різниці Δprice (лаги 8, 15, 20, 29, 34)



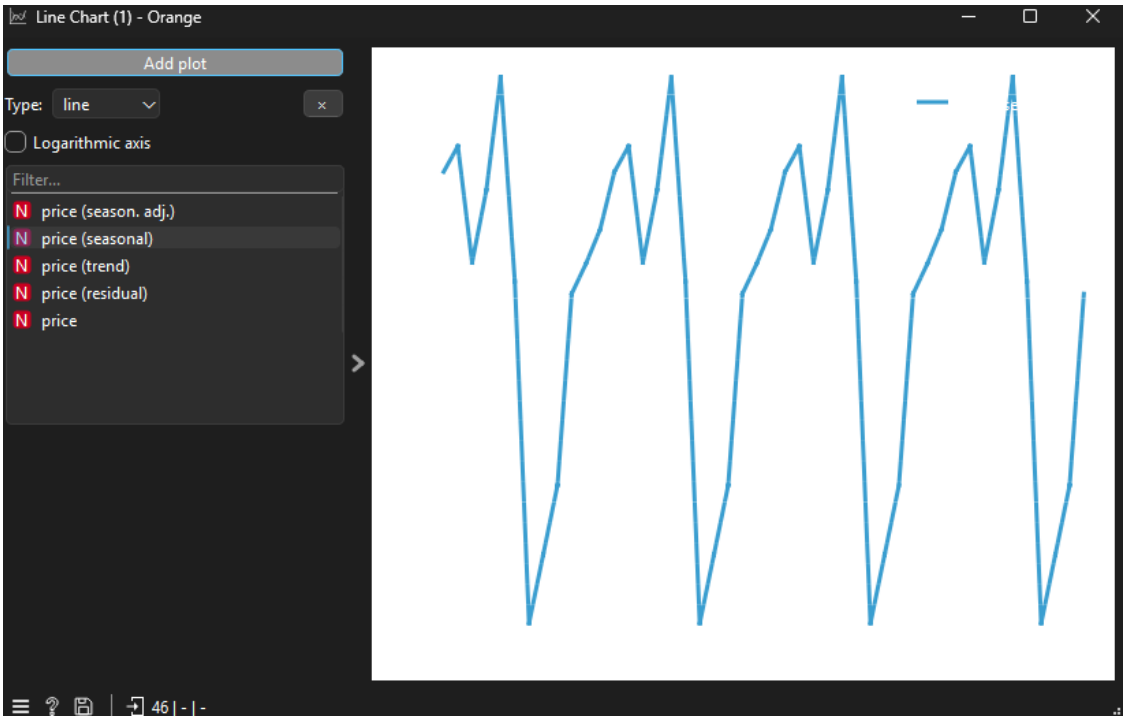
Correlogram: PACF першої різниці Δprice (лаги 8, 14, 20)



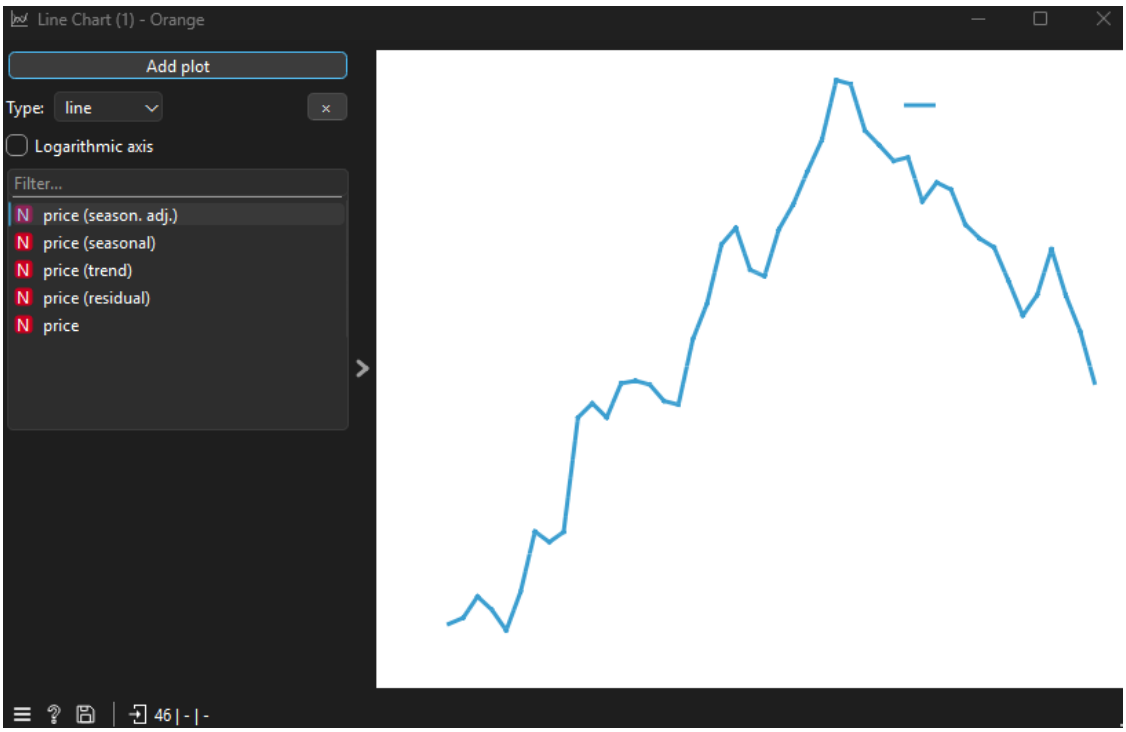
Seasonal Adjustment на часовому ряді з date: помилка «variable date is not in domain»



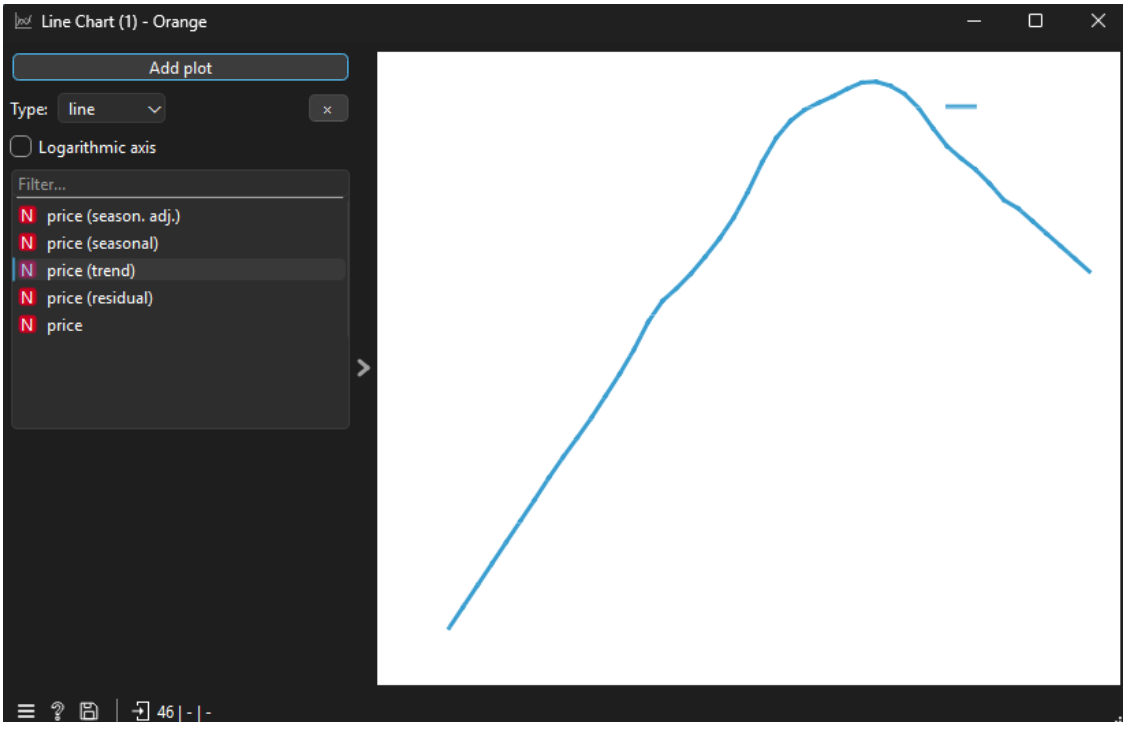
Seasonal Adjustment → Line Chart: price (seasonal) — однакова хвиля щороку



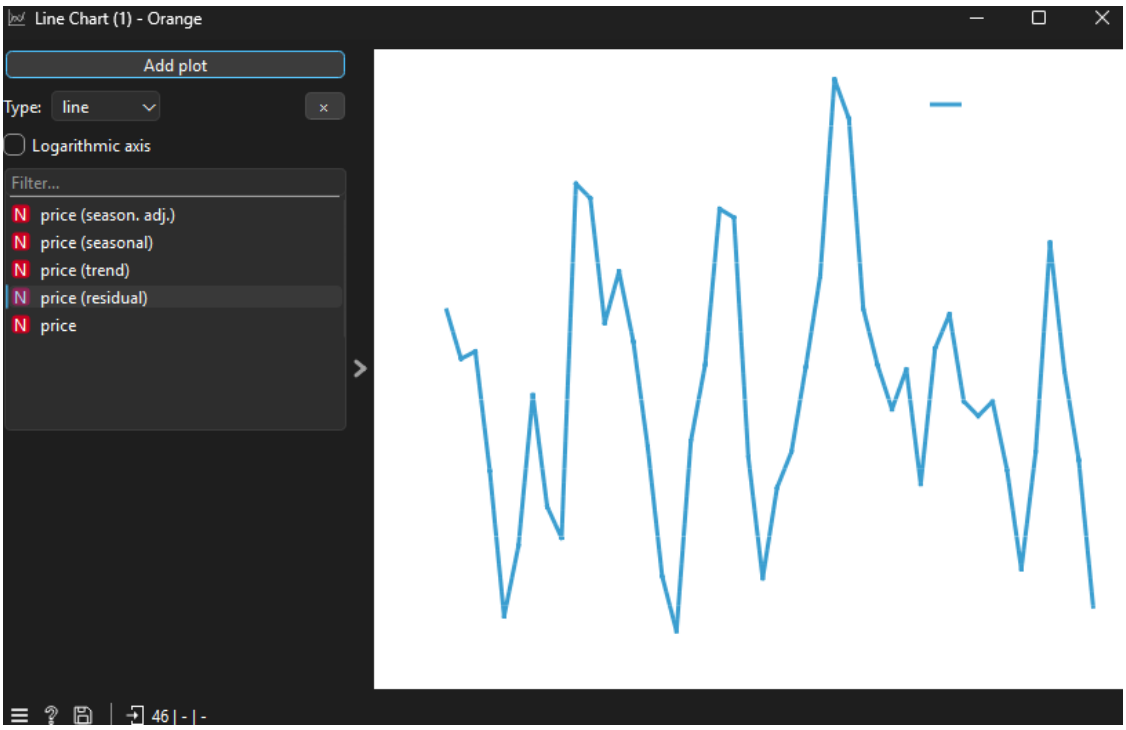
price (season. adj.) — сезонно скоригований ряд



price (trend) — тренд класичного розкладу



price (residual) — залишок



Висновок до завдання 12:

Корелограми. Значення автокореляцій в Orange збігаються з statsmodels до третього знака, бо віджет викликає ту саму функцію `acf`: наприклад, ACF ціни на лагу 24 дорівнює −0.441 в обох середовищах, а відтворення логіки віджета в Python (код вище) дає рівно ті стовпчики, що на знімках. Однак **висновки про значущі лаги не збігаються**. Віджет Correlogram (Timeseries 0.6.3) відображає лише локальні піки |r| — лаги, де значення за модулем більше, ніж у трьох сусідніх з кожного боку. Тому лаги 1–4 вихідного ряду (0.94; 0.86; 0.78; 0.70), на яких ґрунтувався висновок про нестаціонарність, і лаг 1 першої різниці (0.30 — єдиний значущий у statsmodels) в Orange не показано. Межу значущості Orange рахує як одну сталу лінію лише з показаних піків (±0.34 для ACF ціни), тоді як у statsmodels межа Бартлетта розширюється з лагом. Через це лаг 24 вихідного ряду (−0.44) в Orange виглядає значущим, а в statsmodels ні (межа ±0.94). Ця від'ємна кореляція — наслідок форми ряду з горбом (ціни з різницею два роки лежать по різні боки від середнього), а не реальна закономірність. Для першої різниці всі показані Orange лаги (8, 15, 20, 29, 34 для ACF; 8, 14, 20 для PACF) лежать у межах значущості. Отже, з корелограми Orange не можна обрати порядок моделі ARIMA, хоча самі обчислення ті самі.

Сезонне коригування. Віджет Seasonal Adjustment (Timeseries 0.6.3) видавав помилку «variable date is not in domain», якщо на вхід подавати часовий ряд із часовою змінною date. Віджет вирізає з таблиці лише обраний стовпець, але далі звертається до часової змінної, якої у вирізці вже немає. Обхід: подати віджету таблицю без стовпця date (File → Select Columns → Seasonal Adjustment). Класичний розклад використовує лише порядок значень, тому на результат це не впливає.

Orange використовує класичний адитивний розклад (seasonal_decompose), а не STL. Його сезонна складова однакова в кожному році: на графіку та сама хвиля повторюється чотири рази, з максимумом у липні (+2.2 грн) і мінімумом у вересні (−3.2 грн). У STL сезонна складова може змінюватися з року в рік, і її профіль, усереднений за роками, має максимум у квітні та мінімум у вересні. Профілі обох методів подібні (кореляція 0.93: березень–липень вищі за тренд, вересень–листопад нижчі), але амплітуда класичного розкладу менша (5.4 проти 6.4 грн). Тренд класичного розкладу — ковзне середнє, тому для перших і останніх 6 місяців його лінійно добудовано, і на графіку видно прямі відрізки. Сезонно скоригований ряд майже не відрізняється від вихідного: тренд домінує (F_T = 0.97), що узгоджується з висновком завдання 4.

Завдання 13. Модель і оцінювання

Додайте `ARIMA Model` з порядком **(1, 1, 1)** та `Model Evaluation`.

У віджеті **Model Evaluation** встановіть **Number of folds = 7** та **Forecast steps = 3**.

У звіті наведіть: 1. знімок таблиці Model Evaluation з усіма метриками; 2. **що станеться, якщо лишити типові налаштування** (20 фолдів × 3 кроки) — наведіть повідомлення, яке видасть Orange, і поясніть, звідки воно береться; 3. які саме 7 фолдів перевіряє Orange — тобто на яких відрізках ряду навчається модель.

> **Підказка.** Orange відмовиться працювати при типових налаштуваннях. Це не збій, а наслідок довжини вашого ряду — порахуйте, якої саме.

```
# Умова з коду Orange (functions.py, model_evaluation):
#   фолди × кроки + максимальний порядок моделі ≤ довжина ряду
max_order = max((1, 1, 1))                    # для ARIMA(1,1,1) це 1
for n_folds in [20, 7]:
    need = n_folds * 3 + max_order
    print(f'{n_folds} фолдів × 3 кроки + {max_order} = {need} '
          f'{"≤" if need <= len(y) else ">"} {len(y)} → {"OK" if need <= len(y) else "помилка"}')
print('Максимум фолдів при 3 кроках:', (len(y) - max_order) // 3)

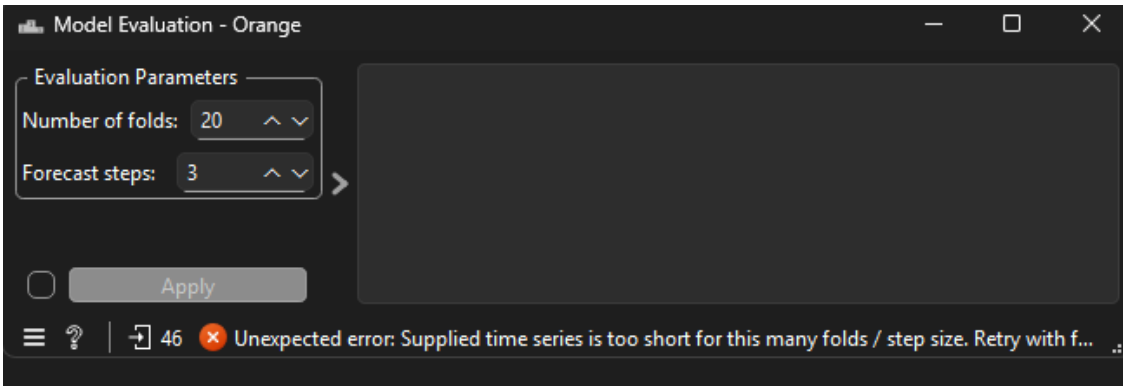
print('\nФолди, які перевіряє Orange (train_end = −3·f):')
for f, (te, idx) in enumerate(folds, 1):
    print(f'  фолд {f}: навчання {y.index[0]:%m.%Y}–{y.index[te-1]:%m.%Y} ({te} міс.)  →  '
          f'прогноз {y.index[idx[0]]:%m.%Y}–{y.index[idx[-1]]:%m.%Y}')
```

```
20 фолдів × 3 кроки + 1 = 61 > 46 → помилка
7 фолдів × 3 кроки + 1 = 22 ≤ 46 → OK
Максимум фолдів при 3 кроках: 15

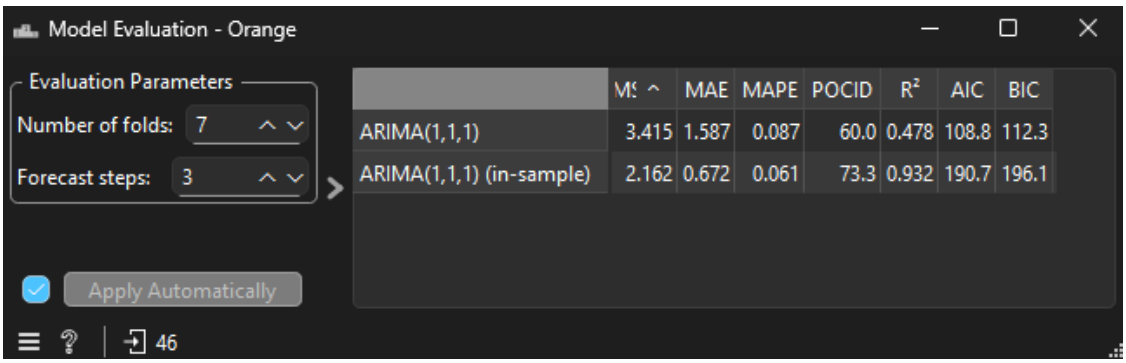
Фолди, які перевіряє Orange (train_end = −3·f):
  фолд 1: навчання 03.2014–09.2017 (43 міс.)  →  прогноз 10.2017–12.2017
  фолд 2: навчання 03.2014–06.2017 (40 міс.)  →  прогноз 07.2017–09.2017
  фолд 3: навчання 03.2014–03.2017 (37 міс.)  →  прогноз 04.2017–06.2017
  фолд 4: навчання 03.2014–12.2016 (34 міс.)  →  прогноз 01.2017–03.2017
  фолд 5: навчання 03.2014–09.2016 (31 міс.)  →  прогноз 10.2016–12.2016
  фолд 6: навчання 03.2014–06.2016 (28 міс.)  →  прогноз 07.2016–09.2016
  фолд 7: навчання 03.2014–03.2016 (25 міс.)  →  прогноз 04.2016–06.2016
```

Знімки екрана Orange (завдання 13)

Model Evaluation, 20 фолдів × 3 кроки: «Supplied time series is too short for this many folds / step size»



Model Evaluation, 7 фолдів × 3 кроки: RMSE 3.415, MAE 1.587, MAPE 0.087, POCID 60.0



Висновок до завдання 13:

До потоку додано віджети ARIMA Model з порядком (1, 1, 1) та Model Evaluation (входи: ряд з Interpolate і модель з ARIMA Model).

За типових налаштувань (20 фолдів × 3 кроки) Orange видає помилку «Supplied time series is too short for this many folds / step size». У коді додатка Timeseries (функція model_evaluation) перевіряється умова «кількість фолдів × кроки + максимальний порядок моделі ≤ довжина ряду». Для типових налаштувань 20 × 3 + 1 = 61 > 46, тобто ряду бракує 15 місяців. Це не збій, а наслідок того, що ряд містить лише 46 спостережень. При 3 кроках максимально можливо 15 фолдів.

З налаштуваннями 7 × 3 (22 ≤ 46) Orange дає для ARIMA(1,1,1): RMSE = 3.415, MAE = 1.587, MAPE = 0.087, POCID = 60.0, R^2 = 0.478. Orange перевіряє 7 фолдів, відлічуючи їх від кінця ряду: фолд f навчається на всіх місяцях, крім останніх 3·f, і прогнозує наступні 3. Найкоротша навчальна частина — 25 місяців (03.2014–03.2016, прогноз 04–06.2016), найдовша — 43 місяці (03.2014–09.2017, прогноз 10–12.2017). Це ті самі фолди, що в Частині 1. RMSE = 3.415 збігається з RMSE, отриманим у Python (3.42), отже, обидва середовища працювали з однаковими даними, моделлю й фолдами. Рядок «(in-sample)» оцінює модель, навчену на всьому ряді, на тих самих даних, тому для порівняння з базисами не використовується.

Завдання 14. Чому числа не збігаються

Порівняйте метрики Orange з метриками, які ви отримали в Частині 1 для **тієї самої** моделі ARIMA(1,1,1) на **тих самих** 7 фолдах.

Для кожної з чотирьох метрик Orange — `RMSE`, `MAE`, `MAPE`, `POCID` — **відтворіть її значення в Python**, тобто напишіть формулу, за якою Orange отримав саме це число, і переконайтеся, що ваш розрахунок збігається з таблицею Orange до третього знака.

У звіті наведіть: 1. таблицю на чотири рядки: метрика · значення Orange · ваше відтворення в Python · формула; 2. **яка з метрик не відповідає своїй назві** — і чим саме вона відрізняється від того, що зазвичай позначають цією абревіатурою; 3. яка метрика збіглася між середовищами одразу і чому; 4. висновок: що саме треба перевіряти, перш ніж порівнювати числа з двох різних інструментів.

> 💡 **Підказка.** Жодна з розбіжностей не є помилкою Orange. Усі формули відкриті — шукайте їх у вихідному коді додатка Timeseries, файл `functions.py`.

```
# Ті самі 21 пара (факт, прогноз) ARIMA(1,1,1) з завдання 7.
# Порядок як в Orange: фолд 1 (10-12.2017), фолд 2 (07-09.2017), …, фолд 7 (04-06.2016)
t = np.array(cv_true['ARIMA(1, 1, 1)'])
p = np.array(cv_pred['ARIMA(1, 1, 1)'])
e = t - p

orange = {'RMSE': 3.415, 'MAE': 1.587, 'MAPE': 0.087, 'POCID': 60.0}     # з таблиці Model Evaluation

# — формули з orangecontrib/timeseries/functions.py —
o_rmse  = np.sqrt(np.mean(e ** 2))                          # корінь із середнього квадрата
o_mae   = np.median(np.abs(e))                              # МЕДІАНА модулів, не середнє!
```

```
o_mape  = np.mean(np.abs(e)) / np.mean(np.abs(t))          # частка: сер.|e| / сер.|y|, не у %
o_pocid = 100 * np.mean(np.diff(t) * np.diff(p) > 0)       # % пар сусідніх точок з однаковим напрямком

# ── «звичайні» означення (як у Частині 1) ──
s_mae  = np.mean(np.abs(e))
s_mape = np.mean(np.abs(e / t)) * 100

tab = pd.DataFrame({
    'Orange': orange,
    'відтворення Orange': {'RMSE': o_rmse, 'MAE': o_mae, 'MAPE': o_mape, 'POCID': o_pocid},
    'звичайне означення (Частина 1)': {'RMSE': o_rmse, 'MAE': s_mae, 'MAPE': s_mape, 'POCID': np.nan},
    'формула Orange': {'RMSE': '√(Σe²/n)', 'MAE': 'median|e|', 'MAPE': 'mean|e| / mean|y|',
                       'POCID': '100·mean[Δy·Δŷ > 0]'}})
print(tab.to_string(float_format=lambda v: f'{v:.3f}'))

# перевірка «до третього знака»
for k, v in {'RMSE': o_rmse, 'MAE': o_mae, 'MAPE': o_mape, 'POCID': o_pocid}.items():
    print(f'{k:6s} Orange {orange[k]:7.3f}  Python {round(v, 3):7.3f}  збіг: {round(v, 3) == orange[k]}')

# ── чому MAE різні: розподіл 21 похибки ──
print('\n|e| відсортовані:', np.round(np.sort(np.abs(e)), 2))
print(f'середнє {s_mae:.3f}  медіана {o_mae:.3f}  — середнє тягнуть угору кілька великих промахів')

# ── чому MAPE різні: не лише одиниці ──
print(f'\nMAPE Orange ×100 = {o_mape*100:.2f} %, а звичайний MAPE = {s_mape:.2f} %')
print(f'Orange: {np.mean(np.abs(e)):.3f} / {np.mean(np.abs(t)):.3f}  (відношення середніх)')
print(f'Звичайний: середнє з 21 відношення |e|/y')

# ── POCID: які пари порівнюються ──
fold_of = np.repeat(np.arange(1, 8), 3)
pairs = pd.DataFrame({'з': fold_of[:-1], 'до': fold_of[1:],
                      'Δфакт': np.diff(t).round(2), 'Δпрогноз': np.diff(p).round(2)})
pairs['тип'] = np.where(pairs['з'] == pairs['до'], 'всередині фолду', 'стик фолдів')
pairs['напрямок вгадано'] = np.diff(t) * np.diff(p) > 0
print('\n', pairs.to_string(index=False))
print(pairs.groupby('тип')['напрямок вгадано'].agg(['sum', 'count']))
```

```
       Orange  відтворення Orange  звичайне означення (Частина 1)       формула Orange
RMSE    3.415               3.415                           3.415            √(Σe²/n)
MAE     1.587               1.587                           2.425          median|e|
MAPE    0.087               0.087                           8.869    mean|e| / mean|y|
POCID  60.000              60.000                             NaN  100·mean[Δy·Δŷ > 0]
RMSE   Orange   3.415  Python   3.415  збіг: True
MAE    Orange   1.587  Python   1.587  збіг: True
MAPE   Orange   0.087  Python   0.087  збіг: True
POCID  Orange  60.000  Python  60.000  збіг: True

|e| відсортовані: [ 0.04  0.18  0.19  0.64  0.66  0.98  1.46  1.48  1.55  1.56  1.59  1.85
  1.94  2.46  2.49  2.84  2.91  3.71  5.46  6.38 10.54]
середнє 2.425  медіана 1.587  — середнє тягнуть угору кілька великих промахів

MAPE Orange ×100 = 8.67 %, а звичайний MAPE = 8.87 %
Orange: 2.425 / 27.971  (відношення середніх)
Звичайний: середнє з 21 відношення |e|/y

  з  до  Δфакт  Δпрогноз             тип  напрямок вгадано
1   1   -1.23      0.03 всередині фолду             False
1   1   -0.87     -0.00 всередині фолду              True
1   2    5.71      2.64     стик фолдів              True
2   2   -0.90      0.01 всередині фолду             False
2   2   -0.90     -0.00 всередині фолду              True
2   3    5.24      2.97     стик фолдів              True
3   3   -1.61      0.04 всередині фолду             False
3   3   -1.07     -0.01 всередині фолду              True
3   4    4.53      0.06     стик фолдів              True
4   4   -0.04      0.03 всередині фолду             False
4   4   -1.30     -0.00 всередині фолду              True
4   5   -0.36     -0.84     стик фолдів              True
5   5    0.87      0.08 всередині фолду              True
5   5   -0.48     -0.01 всередині фолду              True
5   6    8.39      9.42     стик фолдів              True
6   6   -4.48      0.94 всередині фолду             False
6   6   -4.14      0.94 всередині фолду             False
```

```
 6   7   3.31     -9.18     стик фолдів            False
 7   7   0.53     -0.03 всередині фолду            False
 7   7   3.89      0.01 всередині фолду             True
                 sum  count
тип
всередині фолду   7     14
стик фолдів       5      6
```

Висновок до завдання 14:

Порівнювалась та сама модель ARIMA(1,1,1) на тих самих 7 фолдах × 3 кроки. RMSE в Orange (3.415) збігається з Python до третього знака. Це підтверджує, що прогнози в обох середовищах однакові, і всі розбіжності пояснюються лише означеннями метрик. Застосувавши до тих самих 21 пари «факт–прогноз» формули з вихідного коду додатка Timeseries (functions.py), я відтворив усі чотири метрики Orange до третього знака: RMSE = 3.415, MAE = 1.587, MAPE = 0.087, POCID = 60.0.

Метрика, що не відповідає назві, — MAE: Orange рахує медіану модулів похибок (Median Absolute Error, як зазначено в документації функції), а не середнє. Для мого ряду медіана (1.587) значно менша за середнє (2.425), бо середнє збільшують три великі промахи у фолді 6 (до 10.5 грн після піку липня 2016 року), які медіана ігнорує. MAPE в Orange — частка, а не відсоток, і обчислюється як відношення середньої похибки до середньої ціни (2.425 / 27.97), а не як середнє відносних похибок. Тому навіть помножене на 100 значення (8.67 %) не дорівнює звичайному MAPE (8.87 %). POCID — частка пар сусідніх прогнозів, у яких знак прогнозованої зміни збігається з фактичною. Orange рахує його по склеєному списку прогнозів усіх фолдів, тому 6 із 20 пар з'єднують кінець одного фолду з початком іншого, тобто місяці, між якими немає послідовності в часі. Серед 14 пар справжніх сусідніх місяців напрямок вгадано лише в 50 % випадків. RMSE збіглася одразу, бо її означення однакове в обох середовищах.

Висновок: перш ніж порівнювати числа з різних інструментів, потрібно переконатися, що однакові дані, модель і схема перевірки, а потім перевірити означення кожної метрики (середнє чи медіана, частка чи відсоток, спосіб агрегації та порядок обробки прогнозів) за документацією або вихідним кодом, а не за абревіатурою.

Завдання 15. Матриця напрямку зміни

Метрика `POCID` у Orange вимірює, чи правильно модель передбачає **напрямок** зміни ціни. Розгорніть її в таблицю 2×2: рядки — що сталося насправді (зросла / впала), стовпці — що передбачила модель.

У звіті наведіть: 1. заповнену матрицю 2×2 для вашої моделі; 2. перевірку, що частка правильних передбачень збігається зі значенням POCID із Orange; 3. висновок: чи корисна ваша модель для рішення «купувати зараз чи чекати», і чому це **інше питання**, ніж те, на яке відповідають MAE та RMSE.

 Підказка. Ця матриця — часоворядовий відповідник матриці похибок із класифікації.

```
# ── 1. Матриця напрямку саме так, як її рахує POCID в Orange: 20 пар сусідніх точок склеєного списку ──
dt, dp = np.diff(t), np.diff(p)
lab = lambda d: np.where(d > 0, 'зросла', 'впала')
cm = pd.crosstab(pd.Series(lab(dt), name='факт'), pd.Series(lab(dp), name='прогноз'))
print('Матриця (як POCID в Orange, 20 пар):'); print(cm)
acc = np.trace(cm.values) / cm.values.sum() * 100
print(f'Частка правильних: {np.trace(cm.values)}/{cm.values.sum()} = {acc:.1f} %   POCID Orange =
{orange["POCID"]}')

# те саме лише для справжніх сусідніх місяців (без стиків фолдів)
inside = fold_of[:-1] == fold_of[1:]
cm_in = pd.crosstab(pd.Series(lab(dt[inside]), name='факт'), pd.Series(lab(dp[inside]), name='прогноз'))
print('\nЛише пари всередині фолдів (14):'); print(cm_in)
print(f'Частка правильних: {np.trace(cm_in.values)}/{cm_in.values.sum()} =
{np.trace(cm_in.values)/cm_in.values.sum()*100:.1f} %')

# ── 2. Матриця для рішення «купувати зараз чи чекати»:
#    у момент прогнозу відома остання ціна y_T; чи буде ціна через h місяців вищою за неї?
last_known = np.repeat([y.iloc[te - 1] for te, _ in folds], 3)          # y_T для кожного фолду
```

```
act_dir  = lab(t - last_known)
pred_dir = lab(p - last_known)
cm_dec = pd.crosstab(pd.Series(act_dir, name='факт (vs ціна зараз)'),
                     pd.Series(pred_dir, name='прогноз (vs ціна зараз)'))
print('\nДля рішення «купувати зараз чи чекати» (21 прогноз):'); print(cm_dec)
print(f'Частка правильних: {np.trace(cm_dec.values)}/{cm_dec.values.sum()} = '
      f'{np.trace(cm_dec.values)/cm_dec.values.sum()*100:.1f} %')
print('Фактично ціна знизилась відносно поточної у', (t < last_known).sum(), 'з 21 випадку')
print('Наївний прогноз: зміна завжди 0 → напрямку не дає взагалі')

Матриця (як POCID в Orange, 20 пар):
прогноз  впала  зросла
факт
впала        6       6
зросла       2       6
Частка правильних: 12/20 = 60.0 %   POCID Orange = 60.0

Лише пари всередині фолдів (14):
прогноз  впала  зросла
факт
впала        5       6
зросла       1       2
Частка правильних: 7/14 = 50.0 %

Для рішення «купувати зараз чи чекати» (21 прогноз):
прогноз (vs ціна зараз)  впала  зросла
факт (vs ціна зараз)
впала                         11       2
зросла                         4       4
Частка правильних: 15/21 = 71.4 %
Фактично ціна знизилась відносно поточної у 13 з 21 випадку
Наївний прогноз: зміна завжди 0 → напрямку не дає взагалі
```

Висновок до завдання 15:

Матриця напрямку, побудована так само, як POCID в Orange (знаки змін між сусідніми елементами склеєного списку 21 прогнозу, тобто 20 пар):

	прогноз: впала	прогноз: зросла
факт: впала	6	6
факт: зросла	2	6

Частка правильних — 12/20 = 60.0 %, що збігається з POCID у таблиці Orange. Однак 6 із 20 пар з’єднують останній місяць одного фолду з першим місяцем іншого, тобто місяці, між якими немає реальної послідовності в часі. Серед 14 пар справжніх сусідніх місяців напрямок вгадано лише в 7 випадках (50 %). Крім того, прогнози ARIMA в межах фолду майже незмінні (зміни 0.01–0.04 грн), тож напрямок між ними випадковий.

Для рішення «купувати зараз чи чекати» доречніше порівнювати прогноз з останньою відомою ціною. У такій постановці модель правильно вгадала напрямок у 15 з 21 випадку (71 %). Проте правило «ціна завжди знижуватиметься» дало б 62 % (13 з 21), рішень фактично лише 7 (по одному на фолд), а найбільшої помилки модель припустилась на розвороті 2016 року: прогнозувала зростання, коли ціна впала на 7.7 грн. Тому для такого рішення модель ненадійна.

Це інше питання, ніж те, на яке відповідають MAE і RMSE. Вони вимірюють, наскільки прогноз далекий від факту в гривнях, і не розрізняють напрямку помилки; матриця напрямку оцінює лише знак зміни й не враховує її величини. Наївний прогноз найкращий за MAE, але не дає жодної інформації про напрямок. Тобто модель, найкраща за точністю рівня, не обов’язково корисна для рішень, що залежать від напрямку руху ціни.

Загальний висновок

Напишіть 8–12 речень, які відповідають на такі питання:

1. Чи є у вашому ряді сезонність і чим ви це довели?
2. Який метод прогнозування виявився найкращим і чи перевершив він найпростіший базис?
3. Чи знайшли ви точку зміни режиму; якщо так — чи є для неї зовнішнє пояснення?
4. Чому однаково названі метрики у двох середовищах дали різні числа?
5. Чого ви не змогли встановити через те, що ряд має лише 46 спостережень?

Відповідь.

У ряді цін на гречку в Чернігові домінує тренд (F_T = 0.97). Сила сезонності F_S = 0.59 перевищує поріг 0.3, і в усі чотири роки ціна навесні вища за тренд, а у вересні–жовтні нижча, що схоже на ефект збору врожаю; класичний розклад в Orange дає подібний профіль (кореляція 0.93). Проте залежно від параметрів STL F_S змінюється від 0.24 до 0.59, тому сезонність можна вважати лише ймовірною й помірною. Найкращим методом прогнозування виявився наївний прогноз (MAE 2.21 грн на 7 фолдах × 3 кроки). ARIMA(1,1,1) його не перевершила (2.42 грн), як і обрана за ACF/PACF ARIMA(0,1,1) (2.26 грн); коефіцієнти моделі незначущі, а ряд поводиться як випадкове блукання. Точку зміни режиму знайдено в грудні 2014 року: ціна за місяць зросла з 9.30 до 17.29 грн (t = 19.6). Того ж місяця гречка подорожчала на всіх 23 обласних ринках (від +52 % до +99 %), тоді як більшість інших товарів — лише на 0–8 %, тож це, імовірно, шок на ринку самої гречки на тлі загальної девальвації 2014 року; точну причину з цих даних встановити не можна. Однаково названі метрики в Python і Orange дали різні числа через різні означення: RMSE збіглася (3.415), що доводить однаковість прогнозів, тоді як MAE в Orange — медіана, MAPE — відношення середніх у частках, а POCID рахується по склеєному списку фолдів. Через те, що ряд має лише 46 спостережень (3.8 сезонного циклу, 21 прогноз у перевірці), не вдалося надійно встановити наявність сезонності, статистичну значущість точки зміни ($p \approx 0.02$–0.07 за симуляцією) і те, чи різниця між наївним прогнозом і ARIMA не випадкова. Крім того, ряд починається в березні 2014 року, тож вплив подій кінця 2013 — початку 2014 року на ціну виміряти неможливо.

Критерії оцінювання

Бали	Умова
0	Ряд не розкладено або прогноз не побудовано. Робота не зарахована.
2	Виконано лише Частину 1: ряд розкладено й прогноз побудовано, але в Orange аналіз не відтворено.
3	Виконано обидві частини, проте прогноз не зіставлено з наївним базисом, а розбіжність результатів двох середовищ не пояснено.
4	Ряд розкладено на складові; прогноз зіставлено з наївним базисом за ретроспективною перевіркою з рухомим початком; той самий аналіз відтворено в Orange; зіставлено результати двох середовищ і пояснено розбіжність метрик.

Знімаються бали за:

- графіки без пояснення вибору геометрії та шкали;
- числа без письмового висновку;
- прогноз без порівняння з базисом;
- висновок «метрики різні, бо середовища різні» без відтворення формул;
- зошит, який не виконується від початку до кінця.

Декларація про використання штучного інтелекту

Використання інструментів штучного інтелекту під час виконання цієї роботи **дозволено**. Умова одна: ви маєте розуміти кожен рядок зданого коду й кожне написане твердження і бути готовим пояснити їх на захисті.

Заповніть таблицю. Якщо ШІ не використовувався, напишіть «не використовувався» в першому рядку й залиште решту порожньою.

Питання	Ваша відповідь
Які інструменти використано (назва, версія)	Claude (Anthropic), модель claude-opus-5-5, у застосунку Claude; 20–22.10.2026
Для яких саме завдань	Усі завдання 1–15: пояснення понять (STL, ACF/PACF, ADF/KPSS, ARIMA, ретроспективна перевірка, метрики), код для Частини 1 і перевірок у Частині 2, інструкції з побудови потоку Orange (11–13), розбір вихідного коду Orange3-Timeseries (12–14), чернетки висновків, оформлення зошита
Що саме було згенеровано (код / текст / пояснення / виправлення помилок)	Код усіх кодових клітинок; чернетки письмових висновків, загального висновку й переліку помилок у завданні 9; пояснення теорії на прикладах; виправлення помилок під час виконання (NameError у завданні 7, встановлення scikit-learn замість sklearn, обхід помилки віджета Seasonal Adjustment)
Що ви змінили у згенерованому результаті й чому	Додав у завдання 6 побудову таблиці прогнозів, бо без неї код завдання 7 не виконувався; після того як помітив розбіжність, замінив друк значущих лагів у завданні 5 на межі, що збігаються з графіком; потік Orange будував і налаштовував сам, знімки екрана зробив сам
Що з отриманого ви **перевірили самостійно** і яким способом	Запустив увесь код у своєму середовищі й звірив числа; перевірив у Line Chart, що значення в Orange збігаються з Python (лютий 2015 — 17.9167); переконався, що таблиця Model Evaluation збігається з відтворенням у Python до третього знака; звірив стовпчики корелограм Orange з відтворенням логіки віджета; перепитував кожне незрозуміле поняття, доки не міг пояснити його сам
Чи траплялися у відповідях ШІ помилки; якщо так — які	Так: (1) значення кубічного сплайна 23.567 — pandas за замовчуванням будує згладжувальний сплайн, правильне інтерполяційне значення 24.189; (2) первинний висновок до графіка 3 «сезонності немає» суперечив STL-розкладу (F_S = 0.59); (3) друк значущих лагів ACF зі сталою межею не збігався з графіком — помітив я; (4) код завдання 7 посилався на змінну `detail`, не визначену в завданні 6; (5) окремі пояснення були заплутаними, і їх довелося перепитувати

Підтверджую, що розумію весь зданий код і всі зроблені висновки та можу пояснити будь-яку їх частину на захисті роботи.

Підпис: Б.Ю.О Дата: 10/22/2026